This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.DOI

An End-to-End Implemenmtation of a Service-Oriented Architecture for Data-Driven Smart Buildings

LASITHA CHAMARI, EKATERINA PETROVA, AND PIETER PAUWELS.
Department of the Built Environment, Eindhoven University of Technology, The Netherlands

Corresponding author: Lasitha Chamari (e-mail: l.c.rathnayaka.mudiyanselage@tue.nl).

This work is supported by the Brains for Buildings project. Brains for Buildings project received funding from the Dutch Ministry of Economic Affairs and Climate Policy and Ministry of the Interior and Kingdom Relations under the MOOI program.

ABSTRACT

Buildings are connected to multiple information systems such as Building Management Systems (BMS), Energy Management Systems (EMS), Internet of Things (IoT) devices, Building Information Models (BIM), electricity grid, weather services, etc. Modern data-driven smart building software applications demand seamless integration of the above systems and their data. These applications range from monitoring and analytics to data-driven real-time control of building assets and are only efficient if they are reusable, modular and scalable. However, the lack of a system architecture with well-defined Application Programming Interfaces (APIs) that integrates the above systems to provide access to systems and their data poses a significant challenge for developing and deploying such reusable, modular and scalable applications. With the absence of such architecture, the existing applications often rely on ad-hoc tooling, repetitive methods and inefficient data handling practices. In response to the challenges outlined above, this article presents the design and implementation of a system architecture with data-driven smart building applications in mind. The architecture aims to improve the overall reusability, modularity and scalability of smart building applications. The proposed system architecture relies on the Zachman framework and consists of services in five categories, i.e., 1) legacy systems, 2) modern applications, 3) third-party systems, 4) integration software, 5) databases, and 6) user interfaces. The proposed architecture closely resembles the MACH architectural principles: a) Microservices; b) Application Programming Interfaces (API)-first (exposing functionality via APIs), c) Cloud-based components, and d) Headless principles (front-end and back-end logic are decoupled). The proposed architecture is implemented as a proof-of-concept along with three smart building applications, namely 1) a Digital Twin application integrating sensor data with a BIM model, 2) a web application integrating real-time sensor data with semantic graphs, and 3) a data exploratory tool using sensor data, the Brick ontology and Grafana dashboards. Future implementations also include real-time control applications. The proposed architecture and the implementations show a reusable, modular and scalable architecture blueprint that can be used to develop similar architectures for the smart building domain.

INDEX TERMS Building Information Modelling, data-driven buildings, data integration, IoT, linked building data, microservices, reference architecture, REST API, real time data acquisition, solution architecture

I. INTRODUCTION

Smart buildings must fulfill various operational objectives related to energy efficiency, energy flexibility, Indoor Environmental Quality (IEQ), occupant comfort and well-being, etc. Current approaches aiding the fulfilment of these objectives often rely on software programs designed for the purpose. The latter are commonly referred to as smart building applications [1]–[3] and tend to be data-driven. Examples of such applications include Demand Side Management (DSM) using flexible energy sources [4], [5], automated Fault Detection and Diagnosis (FDD) [6], [7], optimizing Heating, Ventilation and Air Conditioning (HVAC) loads [5], [8],

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

[9], Digital Twins [10]–[12], etc. These applications require many supporting services such as data cleaning, aggregation, extract-transform-load (ETL) tools to gather and process data from different sources, etc. Since the above data are heterogeneous, integration is required and essential. Furthermore, system integration is also required because the systems that produce this data are also diverse and use different technologies. On top of these back-end services are also the various dashboards and web applications used to visualize and convey useful information to the end user. When it comes to novel data-driven smart building applications, buildings cannot only rely on the limited capabilities of traditional Building Management Systems (BMS).

Such smart building applications rely on operational and contextual data from various sources and systems. Operational time series data constitutes data from IEQ indicators, valve positions, faults, alarms, event logs, etc. These originate from systems such as BMS, EMS, weather services, etc., and are the primary input for any data-driven application that aims to optimize the operational efficiency of buildings. These data are mostly structured and the technologies for handling this type of data such as time series databases, real-time data streaming engines, and big data frameworks are matured and therefore straightforward to implement.

On the other hand, Building Information Models (BIM), technical drawings, Process and Instrumentation Diagrams (P&ID), spreadsheets, and manuals contain a lot of information about the configurations and relationships between different systems and their components in a building. These constitute the contextual building data (metadata). Compared to operational data, these data are mostly unstructured. There is a growing interest on using semantic technologies such as ontologies to effectively manage and utilize these contextual building data in the architecture engineering and construction industry [13]. Despite the greater benefits, the technologies to harness and integrate this contextual data with other data are not equally matured.

However, extracting, labelling, organizing, sharing and integrating building data for various applications is not straightforward [14]. Many applications are also developed and deployed as standalone services without allowing them to be reused or linked with other applications. This is mainly related to the absence of a well-designed software architecture and well-defined Application Programming Interfaces (APIs) to access systems, data and services. That has led to the use of ad-hoc procedures and tools when developing data-driven applications [15], [16], which have limitations when it comes to reusability, modularity, and scalability [16]. Therefore, smart buildings need to rely on a well-defined system architecture and APIs that enable both the integration of diverse systems and datasets and the orchestration of various services.

Each building is unique, given the diversity of the building stock, system vendors, and technologies. Therefore, developing a system architecture representing each individual building is impossible. Here, an architectural pattern can be useful to represent a generic system architecture for a given building. These common patterns are also identified as a reference architecture, a blueprint or an architecture pattern. These patterns can support the creation of suitable instances of system architectures for a given domain by introducing reusable knowledge and best practices. Examples of such patterns are already widely available in the software architecture domain. For instance, Chanaka [17] presents such patterns for the transportation, telecommunication, retail, education, automotive, healthcare, power & energy and hospitality industries. Although some features of these architectures are common for smart buildings, they cannot be directly transferred to the smart building domain. There are few established architecture patterns available for smart buildings in the literature. Most existing ones are either too abstract, largely conceptual, do not follow well-established architecture frameworks or patterns, not vendor-agnostic or a combination of the above. The absence of such architecture hinders the development of reusable, modular and scalable applications for buildings. In turn, the achievement of the operational objectives remains unrealistic.

This article presents an architecture for smart buildings aiming at developing data-driven applications in a reusable, modular and scalable manner. Two major focus areas are data and system integration because those are the critical yet less investigated aspects of smart building architecture. Building data and systems are also unique for the building ecosystem compared to other domains such as transportation, telecommunication, etc. The design of the proposed architecture relies on three main inputs:

1) Understanding the smart building system architecture with the help of the Zachman architecture framework [18].
2) Deriving functional and non-functional requirements based on interviews with building stakeholders, a literature review on smart building applications, and experience from existing smart building application development efforts [19].
3) Investigating already available, well-tested architecture patterns in the Information Technology (IT) industry [17], [20].

The article shows a proof-of-concept implementation of an instance of the proposed architecture in three smart building applications. As such, the objectives, and therefore, the contribution and novelty of this paper is three-fold:

Objective 1: Develop an architecture for data-driven buildings that is informed by the Zachman architecture framework, the state-of-the-art in the scientific domain and existing architecture patterns. This architecture satisfies the identified functional and non-functional requirements of a data-driven building. The proposed architecture also aligns with the MACH architectural principles [21] and relies on *a) Microservices; b) Application Programming Interfaces (API)-first (exposing functionality via APIs), c) Cloud-based components, and d) Headless principles (front-end and back-*

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

end logic are decoupled).

The artefacts of the architecture are logically grouped into five components for better understandability and separation of concerns. The five components are 1) legacy systems, 2) modern applications, 3) third-party systems, 4) integration software, 5) databases, and 6) user interfaces. The system architecture is further developed based on previous results from integrating BIM, BMS and IoT data on the web [22] and is continuously updated based on new use cases in smart buildings.

Objective 2: Demonstrate how to integrate semantic technologies with the smart building architecture, thereby effectively reusing existing ontologies.

Objective 3: Implement an instance of the architecture and demonstrate the implementation in three use cases, thus providing an *end-to-end solution*. These three applications include 1) a Digital Twin application integrating sensor data with a BIM model, 2) a web application integrating real-time sensor data with semantic graphs, and 3) a data exploratory tool using sensor data, the Brick ontology and Grafana dashboards.

The remainder of this article is organised as follows. Section II discusses the background of software architectures and the related works in software architecture development for smart buildings. Section III presents the research method relying on six steps towards designing and implementing the proposed architecture. Section IV highlights the results relative to the six methodological steps and includes the architecture, its implementation and the use cases. Finally, Section V concludes the article and outlines future work.

II. BACKGROUND AND RELATED WORKS

This section investigates the background, terminology and related works in the software architecture domain. The study then extends towards software architectures for smart buildings. We discuss the smart building domain and how the available software architectures are applied to the smart building domain. The section concludes with the identified research gaps and the main contributions of the paper.

A. SOFTWARE ARCHITECTURE, ENTERPRISE ARCHITECTURE, ARCHITECTURE PATTERNS, REFERENCE ARCHITECTURE AND SOLUTION ARCHITECTURE

The definition of architecture in this context is “the set of principal design decisions about the system” [23]. Similarly to how architectural principles in the built environment help develop complex buildings, software architecture plays a crucial role in developing complex software systems. It provides a structured approach to designing, planning, and organizing the various components and modules of the software application. The architecture is considered the backbone of a successful software system [24]. When applied to a broader enterprise scope, it is referred to as enterprise architecture [20], [25], which captures both business stakeholders and technical IT aspects.

Software architectures that are useful and applicable to specific domains or systems are recognized as architectural patterns and reference architectures that provide the blueprint for creating a suitable architectural solution [17], [23], [26]. Chanaka [17] defines the implementable version of an architecture as a “solution architecture”, which also has three components: business, technical, and deployment. Fig. 1 shows the relationship between an enterprise and a solution architecture. As such, one enterprise architecture enables many different solution architectures (n) for different use cases. According to TOGAF [27], a full-scale enterprise architecture consists of four types of architecture: business architecture, data architecture, application architecture and technology architecture, which are considered subsets of the overall enterprise architecture. These subsets also become different viewpoints for the different stakeholders. In practice, the different stakeholders (e.g., building owners, engineers) have different interests, and the different architecture views help define access rights and reduce inefficiencies and errors [28].

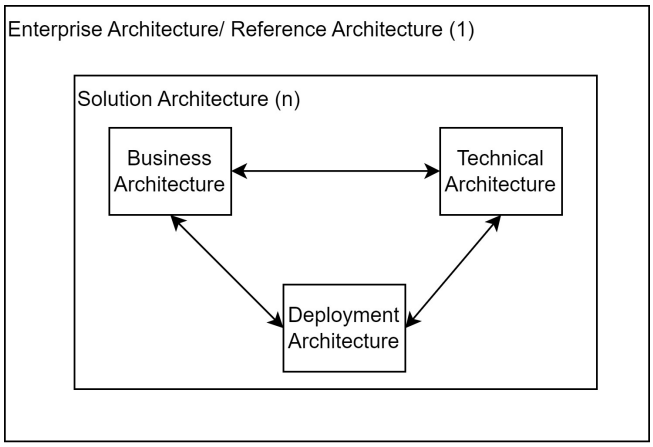

FIGURE 1. Relationship between enterprise architecture and solution architecture according to Chanaka [17]

B. ARCHITECTURE FRAMEWORKS

There are various architecture frameworks that guide the design of a software architecture. ISO/IEC/IEEE 42010 defines an architecture framework as *"conventions, principles and practices for the description of architectures established within a specific domain of application and/or community of stakeholders"*. The two most commonly used frameworks are the Zachman framework [29] and The Open Group Architecture Framework (TOGAF) [27].

The Zachman framework, introduced in 1980, provides the taxonomy for modelling an enterprise. It helps arrange the requirements and the reasoning behind them from 36 different perspectives from two dimensions, as shown in Table 1. The Zachman framework has been used as a framework in several academic and industrial settings, including [30]–[33] and [34]. This framework provides a clear image of the needs and responsibilities of all stakeholders involved in the

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

TABLE 1. Zachman Architecture Framework [29]

Perspective	Data (What)	Function (How)	Network (Where)	People (Who)	Time (When)	Motivation (Why)
Scope (Planner)	List of things important to the business	List of processes the business performs	List of locations in which the business operates	List of organizations important to the business	List of events significant to the business	List of business goals/strategies
Business Model (Owner)	Semantic model	Business process model	Business logistics system	Work flow model	Master schedule	Business plan
System Model (Designer)	Logical data model	Application architecture	Distributed system architecture	Human interface architecture	processing structure	Business rule model
Technology Model (Builder)	Physical data model	System design	Technology architecture	Presentation architecture	Control structure	Rule design
Detailed Representations (Sub-contractor)	Data definition	program	Network architecture	Security architecture	Timing definition	Rule specification
Functioning Enterprise	Data	Function	Network	organization	Schedule	Strategy

development of an enterprise architecture. Nevertheless, it does not provide a method for creating an architecture.

TOGAF is another industry standard that assists in the acceptance, production, use, and maintenance of enterprise architectures. It provides a set of definitions, a process for creating an enterprise architecture, and tooling to support the development [35]. Examples of using TOGAF as a reference are demonstrated in [35]–[37] and [38].

C. ARCHITECTURE DESCRIPTION

Architecture Description (AD) serves as a means to express and communicate the system architecture [24]. There are several established modelling languages for this purpose. The Unified Modeling Language (UML) [39] is the industry standard for specifying, visualizing, constructing, and documenting software system artefacts. Additionally, ArchiMate [40], published by The Open Group, is another notable language. These visual languages also provide a standardized set of icons that facilitate unambiguous communication of architectural concepts. Furthermore, various authors also employ custom icons and languages tailored to their specific needs. Hence, an architectural language can take many forms, ranging from informal box-and-line notations to more formal Architecture Description Languages (ADLs) [41]. In the end, the focus is to provide the necessary information to stakeholders, enabling effective decision-making and understanding of the architecture [41]–[43].

D. CHOOSING AN ARCHITECTURE PATTERN

One way to look at architecture patterns is by domains. These patterns are available for different domains, e.g., automotive industry, healthcare, banking, etc., and result from the experience and best practices of software architects [17]. Patterns provide a starting point when designing a new architecture for a given use case. Leveraging or adapting an existing architecture to suit one's specific requirements offers numerous advantages. One key benefit is the significant reduction in time spent reinventing solutions that have already been thoroughly tested and proven effective in various enterprises. By doing so, one can utilize a well-tested and fail-safe architecture that incorporates best practices [17].

Another way to look at architecture patterns is according to their underlying software technology. As such, Richards [44] provides two major categories for architecture patterns: monolithic (single deployment units) and distributed (multiple deployment units, usually consisting of services). Layered architecture (n-tier), microkernel architecture, and modular monolith are considered monolithic architecture patterns, whereas event-driven, microservices, service-oriented and space-based (cloud-based) architectures are considered distributed [44]. Deciding which architectural pattern to use depends on how each pattern's strengths and weaknesses will impact the problem at hand [44]. This can be solved by analyzing the common characteristics of these software architectures. Table 2 provides a summary of strengths and weaknesses of software architectural patterns as expressed by Richards [20], based on the following criteria:

1) Overall cost: The cost for design, development and running a system architecture.
2) Agility: The capability of a system architecture to adapt to constantly changing requirements.
3) Simplicity: Avoiding unnecessary complexity by using fewer components, services and repositories.
4) Scalability: Ability to grow system capacity over time with the number of users or requests increase.
5) Fault tolerance: Ability of the system to continue to function in an event of error in a component.
6) Performance: The amount of time it takes for the system to process a business request.
7) Extensibility: The ease in which a system can be extended with additional features and functionality.

Based on the above, it can be seen that some architecture patterns offer high scalability, but this comes with an increased overall cost. Other architecture patterns can be simple to develop but not very scalable or extensible. Most importantly, there is no single architecture that outperforms the rest, and understanding each pattern's characteristics, strengths, and weaknesses is necessary to choose

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

TABLE 2. Comparison of architectural characteristics adapted from Richards [20]. One star means the particular architecture characteristic is not well supported, whereas five means it is well-suited for that particular architecture characteristic.

Aspect	Layered (Mono-lith)	Event-driven	Micro-services	Cloud-based
Overall Cost	●	●●●	●●●●●	●●●●●
Agility	●	●●●	●●●●●	●●
Simplicity	●●●●●	●	●	●
Scalability	●	●●●●●	●●●●●	●●●●●
Fault Tolerance	●	●●●●●	●●●●●	●●●
Performance	●●●	●●●●●	●●	●●●●●
Extensibility	●	●●●●●	●●●●●	●●●

the one that meets specific business needs and goals. The state-of-the-art in the area generally discusses monolith, service-oriented (SOA) and microservices-based ones [20], [45]–[49]. A monolithic architecture denotes the traditional method of creating a software program as a single, unified entity, functioning independently of other applications. One notable advantage of adopting a monolithic architecture is the accelerated development pace it affords, attributed to the straightforwardness of consolidating all functions within a single codebase. Furthermore, monolithic architectures exhibit higher performance, outperforming that of microservices, due to the absence of network communication overhead between separate services [20]. However, the drawbacks of a monolithic architecture become apparent in various aspects, particularly in terms of extensibility, agility, fault tolerance, and scalability.

In contrast to monolithic architecture, Service-Oriented Architecture (SOA) is a modern architectural concept that emphasizes loose coupling, reusability, interoperability, agility, and efficiency. The key principle of SOA involves breaking down each business process into smaller, modular blocks of tasks and functions known as services [50]–[53]. To facilitate the integration of various services, SOA often employs an Enterprise Service Bus (ESB) [54], [55]. ESB acts as a central hub, enabling seamless communication between different services and providing a standardized approach to message routing, transformation, and mediation.

The microservice architecture is the newest architecture pattern. Some studies also consider microservices as a subset or a successor of SOA [17], [48] with better performance. Fig. 2 illustrates the basic topology for the microservices architecture style. It consists of individual, specialized services deployed independently and accessed through an API gateway. As the application grows, microservices can be updated in an isolated manner and deployed without affecting the rest of the applications, scoring the highest in agility and extensibility. In this architecture style, each service is responsible for managing its data. Communication between services is straightforward and efficient because of its request-response and event-based message styles [48]. In the microservices architecture, extending functionality is often a straightforward process that includes creating a new

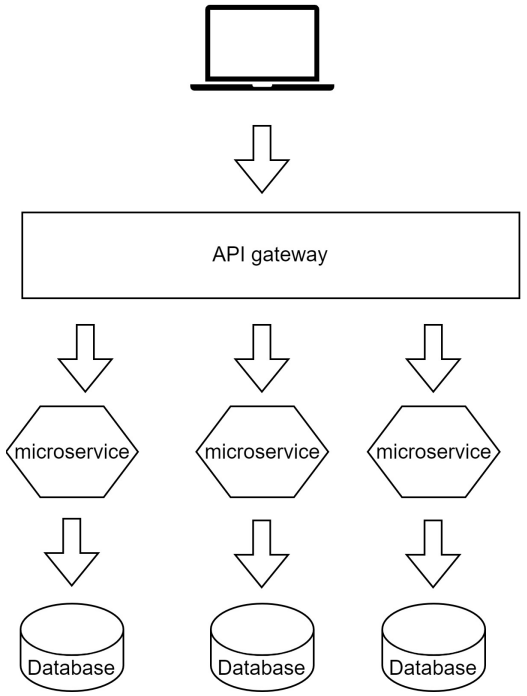

FIGURE 2. The basic topology of the microservices architecture adapted from Richards [20]

service, encapsulating it within a container, establishing an API endpoint, and deploying the service [44]. This makes horizontal and vertical scaling easier compared to monolith or SOA architecture, which explains the higher score in scalability and extensibility in Table 2. Due to the large number of microservices available in a complete solution, managing the testing, deployment, and monitoring of such a large number of services are done using containerization technologies such as Docker and service orchestration and management platforms such as Kubernetes. These artefacts make the microservice architecture more agile and scalable, but also reduce the simplicity and performance due to latency in the communication (network latency, security latency, and data latency) [20] between remote services as depicted in Table 2.

E. DATA-DRIVEN BUILDING DOMAIN

Traditionally, there were few software applications besides a BMS in a building. Therefore, software architecture was not very prominent in the building domain. Nowadays, the constantly growing objectives related to energy efficiency, energy flexibility, IEQ, occupant comfort, well-being, etc., lead to the rapid development of related software applications.

Smart building applications range from monitoring to data-driven control of built assets. Some cutting-edge applications include FDD [56], [57] in HVAC systems, performing Model Predictive Control (MPC) [5], [9], [58] for optimizing energy usage in buildings, or Digital Twins [10]–[12] which are virtual replicas of the physical assets in a building providing

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

ample opportunities for real-time monitoring, identifying and locating faults, etc.

The first requirement for data-driven applications is the ability to integrate various systems. This is mainly due to two reasons, i.e., to utilize their data and to establish live communication with the controllers. Such systems may include BMS, EMS, IoT devices, BIM, and external services for weather and grid price signals. In this regard, various tools aiming to acquire data from heterogeneous sources have been proposed in the literature. For example, Prakash et al. [5] and Zhang et al. [58] demonstrate the ingestion of time series data from on-premise devices, such as thermostats, battery storage, and IoT devices, using drivers that conform to specific device protocols such as Modbus or BACnet. They also ingest weather data from weather APIs and publish it to the WAVEMQ message queue. Applications communicate with the message queue to get the needed data. Agarwal et al. [15] utilize data connectors - programs that retrieve data from the sensor network and write data back for actuation. These connectors are customized for the data source, and sending an HTTP POST command actuates the underlying device. This approach enables bidirectional communication, which is essential for data-driven building controls. Prakash et al. [5] present a similar approach, where set points are published to the message bus WAVEMQ, and the drivers receive the set points to actuate the necessary changes. Based on this literature, protocol adoption is an essential part of data ingestion since no single protocol dominates the implementations. Related work in [1], [5], [59]–[61] employs different concepts such as drivers, middleware, proxies [62], or adapters. Yet, the concept remains the same and follows a similar logic.

The second requirement is the data integration. With a lot of operational and contextual data coming from various systems, the ability to link and integrate them with applications is crucial. To that end, a large body of recent research explores the creation and use of domain ontologies. An ontology is a structured and formalized model representing the knowledge within a given domain [63]. It encompasses the formalization of concepts, relationships, classes, and attributes within the domain. In an ontology, concepts are represented as nodes in a graph data structure, while the edges represent the relationships between them. Ontologies in the building domain such as the Building Topology Ontology (BOT) [64], [65], Semantic Sensor Networks (SSN) [66], Sensor-Observation-Sampling-Actuation (SOSA) [67], Building Automation and Control Systems (BACS) [68], Brick [2], [69] and Haystack [70] are used to instantiate a standard semantic description of heterogeneous systems such as BMS, BIM and IoT. These ontologies not only standardize the semantics but also enable linking above heterogeneous domains. The use of ontologies in this context has been demonstrated in a variety of use cases, including BIM and IoT integration [22], context-aware control of mechanical systems [71], [72], automating KPI calculation for building performance [73], automatic setup of FDD for BMS [74], etc.

F. SOFTWARE ARCHITECTURES FOR THE SMART BUILDING DOMAIN

A number of reference architectures are available in the literature for different domains. For instance, the Industrial Internet Consortium (IIC) presents the Industrial Internet Reference Architecture (IIRA) [26] for the IoT domain. They identify five disciplines of a typical IIoT (Industrial IoT): control, information, operations, application and business domains. They map the identified disciplines into a three-tier architecture with an edge tier, platform tier and enterprise tier. IIC also provides four architecture viewpoints, namely business, usage, functional and implementation viewpoints.

The AIOTI reference model [75] is another high-level IoT architecture with three layers, including network, IoT and application layers. Compared to IIRA, AIOTI provides less use of architecture frameworks and demonstrates a very high-level representation of the IoT domain. However, IIRA and AIOTI are reference architectures and do not propose any solution architectures.

The BDV Reference Model [76] serves as a common reference framework around big data technologies on the overall IT stack. Here, the primary source of big data is sensor data from an IoT context and actuator interaction in cyber-physical systems. Again, this is a generic high-level reference architecture that does not provide a solution architecture.

Within the scope of big data, another existing architecture in the smart buildings context is the MATRYCS [16]. MATRYCS is a reference architecture for building services utilising six design principles, namely modularity via microservices, cloud virtualization, openness and data sharing, security, no vendor lock-in, and distributed data ecosystem. This high-level architecture consists of four layers: i) infrastructure, ii) governance, iii) processing and iv) analytics. These layers encompass services such as data ingestion, interoperability, data processing, streaming, data harmonization, querying, reasoning and sharing. The authors also propose a solution architecture; however, a demonstration of the applicability of the proposed architecture in a real-world use case is missing.

Mazzara et al. [77] propose an ICT reference architecture for smart buildings containing four layers: hardware, network, management and application & service. This is also a high-level architecture, and a specific solution architecture is not presented.

The BuildingDepot architecture [15] focuses on enabling the implementation of portable data-driven applications on top of the distributed physical resources present in large commercial buildings. Its main principles are scalable data storage, ease of data access, fine-grained data sharing, and access control. The architecture relies on REST APIs for system interaction, as demonstrated in the presented use cases. However, the architecture does not include other essential aspects such as metadata management, integration of external services, or user interfaces.

Chevallier [10] proposes an architecture focusing only on the integration of sensor data with smart building ontologies.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

However, this architecture only describes a part of the smart building ecosystem. Abu-Matar [78] proposes a data-driven reference architecture for smart cities inspired by SOA. However, the result is limited to a UML meta-model for the views, their corresponding elements, and their relationships. These custom views create ambiguity when expressing the different artefacts in the architecture, as opposed to utilizing existing architecture frameworks such as Zachman or TOGAF.

Genkin [79] introduces B-SMART, a reference architecture that facilitates the autonomic optimization of smart buildings. B-SMART is also a high-level conceptual layered architecture, the implementation of which remains to be seen.

Bashir [80] proposed the IBDMA reference architecture, which also uses standard components to ingest data; however, it does not include any insight related to data integration or bidirectional communication. Bashir [81] also provides a metamodel, which can be considered a taxonomy of the concepts around big data and IoT. However, this metamodel does not rely on existing architecture frameworks or semantic technologies, which can be a limiting factor when extensibility is considered. Creating new data models instead of reusing and extending existing ones limits the reusability of such applications.

Bao et al. [59] propose a microservice architecture for IoT and the energy domain in smart buildings. Their architecture includes heterogeneous systems, drivers for protocol adaption, databases, services (such as presence prediction service and HVAC set point resolver) and energy-related applications. They use a message hub to establish communication between services. However, this architecture is also presented at an abstract level and does not have an implementation in real-world use cases.

There are also reference architectures provided by commercial parties, which showcase the different products a service provider can offer to develop an architecture for a data-driven building. Some artefacts of these architectures can be recognised in commercial products from service providers. Microsoft Azure provides reference architectures for various applications. The closest category to smart buildings is the IoT category, which provides 70 different architectures [82]. Amazon Web Services (AWS) also provides 48 reference architectures [83] under the IoT category. Due to the fully cloud-based nature of these architectures, they inherently support characteristics like scalability and fault tolerance. However, these architectures still need the domain-specific components identified from the smart building domain, such as drivers, smart building ontologies, etc. Therefore, they cannot be a direct reference for a data-driven building.

In reality, various systems and services in buildings are individually installed by different vendors at different times, and there is no overarching architecture. This fact is not well-represented in any smart building architectures proposed in the literature. When adopting architecture patterns from the software architecture domain to a building, what exists in a building can be closely represented by the brown-field enterprise architecture illustrated by Chanaka [17] in Fig. 3. This kind of mixed enterprise environment (i.e., brown-field or a blended architecture) [84] is a combination of components from different architecture patterns [17] utilised to realise business needs. In fact, according to Chanaka [17] what is mainly observed in many enterprises is a brown-filed architecture rather than a clean architecture (i.e., based entirely on microservices) [17]. It is possible to design a new architecture with microservice principles in mind (Modern Application Layer in Fig. 3). However, the existing heterogeneous applications (top layer in Fig. 3 also need to coexist with the new architecture. A software architecture aims to integrate existing legacy systems while allowing modern applications to be integrated following the best practices in software architecture patterns.

G. RESEARCH GAP AND CONTRIBUTION

Based on the performed literature review, the following three research gaps have been identified:

1) The existing architectures for smart buildings generally do not consider best practices (e.g., architecture frameworks) and existing architecture patterns. Achieving implementable solutions requires an adaptation of the best practices for the smart building domain, as stated in Objective 1.
2) Smart building research has demonstrated the value of semantic technologies to data integration in smart buildings. However, previous studies have not integrated such technologies as an architectural component and exhibit limited capabilities using ad-hoc tools. Metadata integration plays a crucial role in the architecture and should be implemented in the smart building architecture (Objective 2).
3) Most reference architectures are presented at abstract levels, and the technical details required for implementation are scarce. Therefore, it is important to demonstrate the proposed system architectures with an *end-to-end solution* for better understandability and usability. This study proposes a solution architecture and provides example implementations in three smart building use cases, as highlighted in Objective 3.

III. RESEARCH METHOD

This section discusses the six-step research method associated with designing and implementing the architecture for smart buildings.

Step 1 : The first step is to understand the data-driven control domain of an existing building ecosystem in terms of the requirements coming from key stakeholders. This step builds on previous work investigating the data needs and requirements for smart buildings [85]. The latter were identified based on a series of interviews with relevant stakeholders (building owners and technical solution providers).

This study relies on the Zachman architecture framework to organize the above information as a guideline for designing a system architecture. With every cell containing the relevant artefacts, a comprehensive understanding of the system

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

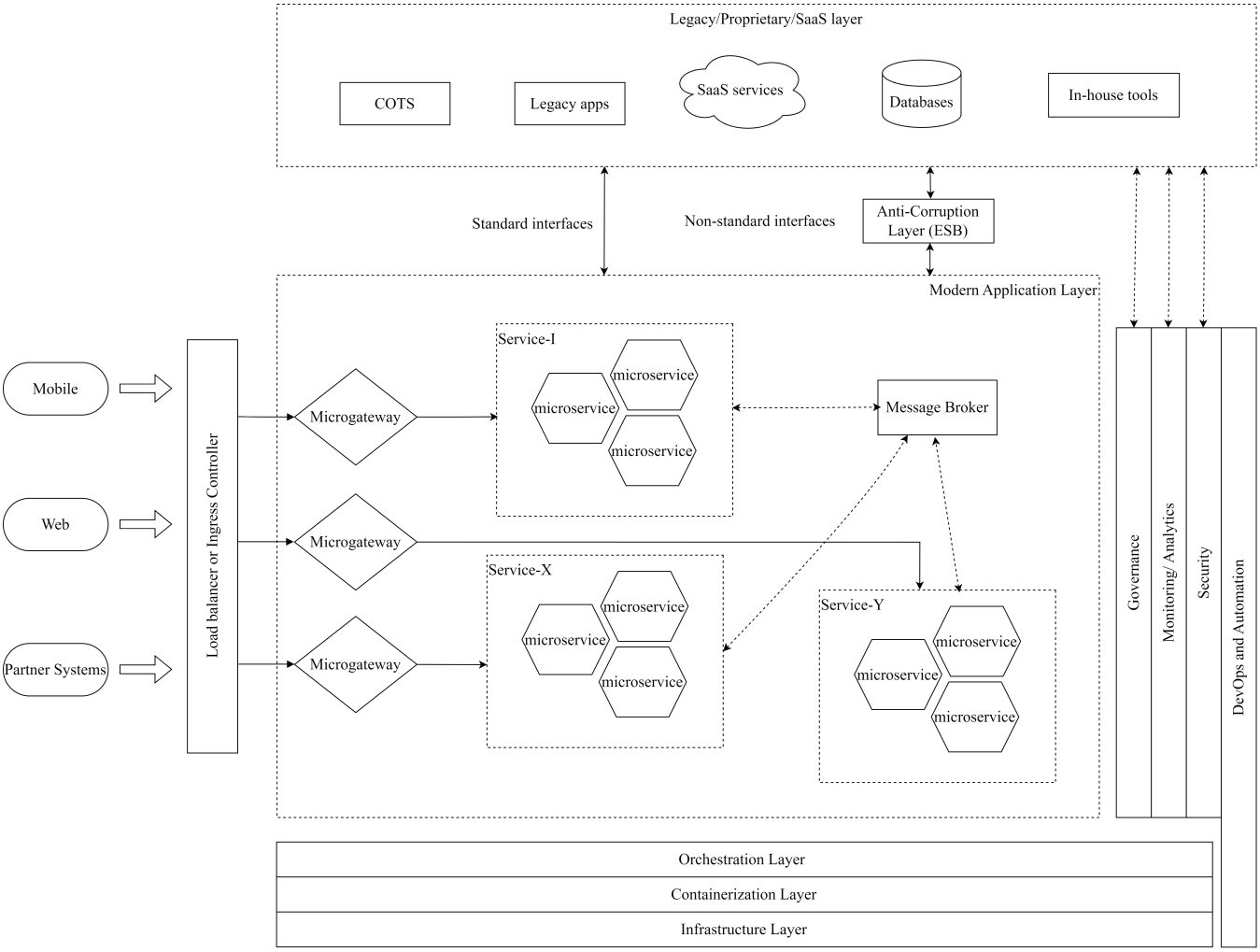

FIGURE 3. Microservices based architecture for a brownfield enterprise where a combination of existing and new applications co-exists to perform a business task (image adapted from Chanaka [17]).

from each stakeholder's viewpoint is achieved. For example, "data" for the business owner is the essential information about the business, while it is rows and columns for the database administrator. Using the Zachman framework helps establish a clear link between the business requirements and their corresponding technical implementations.

Step 2 : The next step defines the functional and non-functional requirements of the system. These requirements are informed by the previously performed interviews and the state-of-the-art review. Defining a set of functional and non-functional requirements helps determine the architecture pattern and identify the necessary software components of the architecture.

Step 3 : This step entails the selection of an architecture pattern suitable for the smart building ecosystem identified in Step 1 and the requirements specified in Step 2. This decision is strongly influenced by the nature of the smart building ecosystem, which is identified as brown-filed in the literature. A smart building already has multiple software systems, and modern applications for data-driven control should be implemented as independently deployed services by connecting with existing software systems.

Step 4 : The next step defines the components of the architecture. This is achieved by i) identifying existing systems in buildings, ii) identifying common components by studying reference architectures and architecture patterns available in other domains, and iii) determining which new or unique components will be needed for the smart building domain based on the requirements from Step 2 and the architecture pattern from Step 3.

Step 5 : This step concerns the development of the architecture. Here, we propose a solution architecture, mainly focusing on the technical architecture. At this stage, the definition of the system architecture for a smart building is complete.

Step 6 : The last step includes the implementation of an instance of the proposed deployment architecture, which is an additional step we take to provide a proof-of-concept for different applications. As seen in the literature review, designing and implementing an architecture is a non-trivial

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

process. It involves several stakeholders, software architects, developers and deployment infrastructure. In this article, we focus on the technical architecture and demonstrate the implementation in three smart building applications.

IV. RESULTS

A. STEP 1 : DATA-DRIVEN BUILDING ECOSYSTEM IN THE ZACHMAN FRAMEWORK

Table 3 illustrates how the data-driven smart building ecosystem is mapped to the Zachman framework. Here, specific artefacts are described with different purposes for different roles. Since Zachman framework is an enterprise architecture design framework, this mapping takes into account six viewpoints from six different stakeholders. All three components of the solution architecture identified from the literature (business, technical and deployment) are placed in their appropriate cell and highlighted in Blue. Since the main focus of this study is the technical architecture, artefacts that are important for the technical architecture are represented in Bold.

B. STEP 2 : FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

In the following section, we present the functional and non-functional requirements of the system architecture. Functional requirements describe what the system should do to support the user actions, while non-functional requirements describe how the system should operate.

1) Functional requirements

1) F1 Ingestion of time series data from heterogeneous systems.
2) F2 Gathering, management, and processing of large volumes of time series data.
3) F3 Gathering, management, and processing of contextual information (metadata) from various data sources.
4) F4 Integrating metadata with the operational data of various devices in a building.
5) F5 Interoperability among several existing devices, services, and applications.
6) F6 Real-time bidirectional communication ability to autonomously control facilities within the smart building.
7) F7 User-centered interfaces for easy information access.

2) Non-functional requirements

Most non-functional requirements are common to a system architecture regardless of domain or architecture pattern. These include performance, scalability, extensibility, security, reliability, and maintainability. In addition, the smart building context also determines the following non-functional requirements:

1) NF1 Reusability of components and services implemented in the architecture for a wide range of smart building applications.
2) NF2 Add new services in a modular way. Each module performs a specific and well-defined function, and these modules can be developed, tested, and maintained independently.
3) NF3 Scalability to manage the increasing amount of devices, data or services in the building.

C. STEP 3 : ARCHITECTURE PATTERN

The aim is to define an architecture for a data-driven smart building, building upon already existing architecture patterns that closely resembles the data-driven building ecosystem. As such, we recognized from the literature that the brownfield architecture presented in Fig. 3 is a good starting point. There is a requirement to support seamless access to various existing sources and their data (F1, 2, 3), and their integration (F4). Included also here are the new services (NF2) to be deployed such as data-driven applications. Multiple services are already available in a building, and the goal is to keep and integrate them with new services (NF2). Based on the literature review, the microservice architecture is more suitable for new applications because its modular approach facilitates scalability, reusability and extensibility (NF1, 2, 3). Furthermore, the need to transport messages between different services (F5, 6) is also fulfilled when using a microservice architecture. This inter-service message transport is essential in sophisticated control strategies like DSM to trigger one service (e.g. calculate set points) based on the output of another service (e.g. building load forecast). This type of coordination is usually achieved by using "event-based messaging" in microservices. As previously mentioned, extending the functionality of a microservice architecture is often a straightforward process, involving creating a new service, encapsulating it within a container, establishing an API endpoint, and deploying the service [44].

D. STEP 4 : IDENTIFYING SOFTWARE COMPONENTS

This section presents the main artefacts that will constitute the architecture. Some components are derived from Fig. 3 based on the literature review, and the others are specific to the data-driven building domain. The list of components is as follows:

1) Existing business applications (legacy systems)
2) New microservices-based business applications
3) Databases
4) Integration software
5) Infrastructure services
6) Shared services
7) User Interfaces

1) Existing business applications (legacy systems)

Examples of existing systems include BMS, EMS, IoT platforms, AMS, dashboard and reporting software, etc. These services can be cloud-based (e.g., IoT platforms, Grafana dashboards), commercial off-the-shelf (COTS) software (e.g., Autodesk Revit) or in-house built applications (e.g. data cleaning tools, data analysis tools). Other useful services include various external services such as weather,

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

TABLE 3. Smart data-driven building ecosystem in the Zachman Framework. Different components of the enterprise architecture are places in suitable cell and highlighted in Blue. Components in Bold are eloborated in the upcoming sections.

Perspective	Data (What)	Function (How)	Network (Where)	People (Who)	Time (When)	Motivation (Why)
Scope (Planner)	**Business architecture** business objectives, strategic goals, high-level requirements for smart building	Strategies for achieving energy efficiency, occupant comfort, and other objectives	Geographical locations of buildings and facilities requiring smart technology	Key stakeholders, decision-makers, facility managers, occupants and beneficiaries	Timelines for implementing architecture, smart technologies, and achieving specific objectives	Justification for adopting smart building technology such as compliance to legislation, occupant satisfaction and well-being, and alignment with business goals
Business Model (Owner)	Business processes related to facility management, maintenance, and operations	Methods and procedures for managing building systems and addressing maintenance issues	Locations where facility management activities are conducted	Roles and responsibilities of facility managers, maintenance staff, third party service providers, and occupants	Timing of regular maintenance tasks, emergency responses, and system updates	Cost savings, benefits to the organization, and occupant satisfaction.
System Model (Designer)	**Functional and non-functional requirements of the system, data processing requirements, logical data models, real-time monitoring needs,** real-time control needs	**Technical architecture**	**Deployment architecture** Locations where system components are deployed (on-premises, cloud, edge)	Development teams, system administrators, technical support personnel	Development milestones, system deployment schedules, software updates	Rationalization for specific technology choices, system performance objectives, and operational efficiency
Technology Model (Builder)	**Software modules, integration interfaces, APIs, hardware components**	Software design patterns, software development methodologies	Physical locations of servers, sensors, and devices where components are deployed	Development teams, integration experts, hardware suppliers	Component development and integration timelines, system maintenance schedules	Component re-usability, modularity and scalabiity, system reliability.
Detailed Representations (Sub-contractor)	**Network infrastructure, IoT platforms, building automation network, data storage and processing technologies**	Network design, protocols, security measures, data encryption methods	Physical and virtual locations of servers, gateways, and data centers	Technology vendors, partners, service providers	Timelines for developing, software integration, and technology upgrades	Justification for technology selection
Functioning Enterprise	**User interfaces, dashboards, reports, control mechanisms**	User experience design, interaction patterns, user training methods	Locations where users access interfaces (desktop, mobile, dashboards)	Facility managers, maintenance personnel, occupants	Operational hours, user activity patterns, maintenance windows	Improved user satisfaction, ease of operation, real-time insights, remote monitoring capability

utility service providers, and charging stations where the data is exposed via their APIs. The integration of these services with the overall system architecture is discussed in the integration software section.

2) New microservices-based applications

As mentioned, the architecture will rely on the microservice pattern when introducing new applications. We identified three types of new applications: i) drivers/connectors for new systems (e.g., IoT drivers, BMS drivers), ii) data-driven applications themselves (e.g., MPC, FDD) and iii) supporting services for data-driven applications (e.g., forecasting services, data aggregation services, data cleaning services, ETL processes, file conversion services such as IFC to XKT). These are illustrated in Fig. 6. These new applications need to communicate with the API gateway via standard interfaces such as HTTP and message queue systems such as Kafka, MQTT, or NATS. This ability to integrate different services when needed is the key to extending the possible applications that can be implemented atop the system architecture without

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

disrupting the existing ones.

3) Databases

The proposed system architecture needs to facilitate different types of data, such as time series data, Resource Description Framework (RDF) graphs of semantic data, object data (such as IFC files, geometry files, PDFs), and other data, such as user and project data. These constitutes structured (e.g., CSV, spreadsheets), semi-structured (e.g., JSON, XML) and unstructured (e.g., PDF, images) data. According to the data type, several databases need to be selected to make data storage and querying more efficient. Time series databases are optimized to store and query time series data, whereas object databases are suitable for storing files. Object storage can also be used to store archived large files and sensor data archives, which can be used for data-hungry applications relying on Machine Learning. User and project details of different applications also need to be saved, ideally in a relational or document database. RDF graphs of building metadata are stored in a graph database. This study relies on four types of databases:

1) Time series database
2) Graph database
3) Blob storage
4) Document database

The logical data models for these four databases are shown in Fig 4. A logical data model entails the conceptual representation of the data without the specific technical details of the database management system.

4) Integration software

Integration software allows software systems developed and designed by different vendors using different technologies to share data. In a microservice–based software platform, integration software can also be deployed as microservices. Standard interfaces such as HTTP, custom drivers/connectors, integration Platform-as-a-Service (iPaaS), and message queue systems such as Kafka, RabbitMQ, or NATS [86] enable the integration of existing services with the overall architecture [17].

Five integration software services, essential for the data-driven building domain, are discussed below.

BMS integration software

The BMS driver is the interface between the BMS and the system architecture. Access to data from the BMS is essential, whether directly connecting to the BMS or downloading the data from the BMS's database periodically to the time series database. Furthermore, a communication link with the BMS controllers needs to be established to send control commands to them based on the implemented smart building applications.

IoT integration software

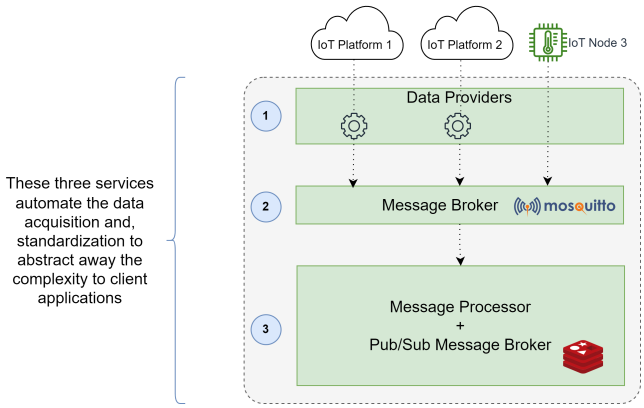

FIGURE 5. IoT integration service to acquire and unify payloads originating from heterogeneous sources [87]

Although the BMS is typically the main source of sensor data, many buildings implement IoT devices as a retrofit. They also provide access to streams of real-time sensor data. There are many such devices, each communicating to their own IoT platform using gateways. Therefore, different connectors are required to access data from each IoT platform. One example is the IoT integration software [87] proposed earlier, where sensor data acquired from different IoT sources (Node 1 of Fig. 5) is published into a central message broker (Node 2 of Fig. 5). A new IoT driver must be introduced whenever a new device needs to be integrated (Node 1 of Fig. 5). These data providers are only responsible for publishing all the messages to the MQTT broker.

Real-time data can be accessed through a message broker, such as Mosquitto, using the MQTT protocol. MQTT is a lightweight, publish-subscribe network protocol that transports messages between devices. The protocol usually runs over TCP/IP. It was designed as an extremely lightweight publish/subscribe messaging transport. MQTT follows a "topic" and "payload" (in JSON) structure to communicate data. Under one topic, it is possible to communicate multiple data points.

In Node 3, an ELT process runs to format the various IoT payloads into a uniform structure, creating a uniform data access for data-driven applications. Here, the logical data model of the outgoing IoT payload is shown in Listing 1. This IoT data is also recorded in a time series database using a suitable ETL process.

```
{
  topic: 'temp/111111111111/pub',
  keys: [
    'timestamp',
    'field-1',
    'nested-field-1.field-1',
    'nested-field-1.field-2',
  ],
  values: [
    '2023-02-12 13:10:55.825397',
    'value-1',
    'value-2',
    'value-3',
  ],
  timestamp: 1676294724521
}
```

Listing 1. Logical data model of the uniform IoT payload

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

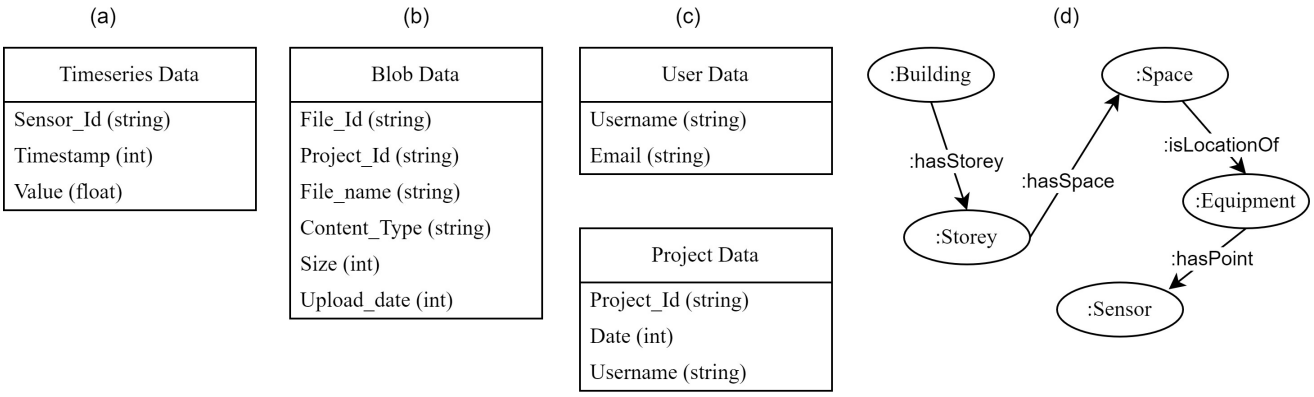

FIGURE 4. Logical data models of a) time series database b) Blob storage c) Document database and d) Graph database

BIM integration software

BIM models are usually developed using commercial software like Revit. These models consist of architecture, structural and Mechanical Electrical and Plumbing (MEP) models from the design stage. If integrated into the operational stage of the building, they can bring huge benefits. To integrate these models, one can use commercial off-the-shelf products like Autodesk Tandem for creating Digital Twins. The downside, however, is the vendor-lock-in and less flexibility and limited extensibility because of their closed APIs. Here, we propose an alternative method using emerging openBIM software. We first export the IFC file from the BIM authoring software and then utilize the IFC files using libraries like IFC.js [88] or xeokit [89] to create front-end web applications enabling interaction with the BIM model. The Global Unique Identifiers (GUID) of elements in the BIM model can be used to link the BIM objects with their operational data via APIs. The BIM models are stored in the object storage database.

Metadata integration software

We use smart building ontologies to create an extensible semantic graph of the building (also called a metadata schema). Using ontologies such as Brick and Haystack, this metadata schema is becoming an essential artefact for BMS systems. The graph carries the physical, logical and virtual assets in buildings and the relationships between them. Furthermore, it contains references to external systems such as time series databases and IFC models, enabling the automatic querying and retrieval of time series data associated with the instances in the metadata schema.

Generating this kind of metadata schema requires converting structured or unstructured metadata from various sources into subject-predicate-object triples that conform to the selected ontology. Conversion of the available metadata in different systems like BMS and BIM from different formats like CSV or IFC into a metadata schema requires multiple drivers, depending on the source system. Here, we show two essential metadata integration softwares.

1. IFC to RDF driver: Creating a metadata schema from an IFC model requires a purpose-built converter to understand the available resources and map them to a desired ontology. The converter in [90] transforms the IFC statements into RDF statements conforming to the PROPS, BOT and MEP ontologies.

2. BMS and IoT Metadata driver: Semantics are extracted from the BMS metadata sources like data dictionaries and device specifications. When identified with their relationships, these can be parsed using a tool like Brick-builder [91] to create an RDF graph. Brick and SSN ontologies can be used to describe the BMS and IoT domains. Similar to BMS device IDs, IoT data streams are identified by their *topics*. There is no universal way to define a topic in MQTT; it uses best practices using a combination of the location and MAC address of the device. Therefore, topics can be identifiers for real-time data and can be included in the semantic graph. These topics can be included using a `brick:hasTimeseriesId` relationship and act as the unique identifier of the real-time sensor data of an IoT node. Finally, these graphs are stored in a graph database.

API Software

The final component of the integration software is the API. After creating databases and core services, these are exposed to internal and external consumers via API software. The API provides a standardized way of accessing data towards its users with REST or GraphQL over HTTP. In addition, API gateways add security, access control, and monitoring at API level, avoiding the necessity to implement these at each service's individual level.

5) Infrastructure services

These include the underlying containerisation, container orchestration, and infrastructure layers of the platform.

6) Shared services

The functionality that is common across all the services will be implemented as shared services. This includes security, governance, monitoring, and automation.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

7) User Interfaces

Included here are user interfaces such as web applications, dashboards or charts that can provide meaningful ways to interact with the data and the necessary access to the end user.

E. STEP 5 : SOLUTION ARCHITECTURE

The above software components constitute the solution architecture. In the literature review, we noted that there are multiple views for this architecture such as: business, technical and deployment. However, this study focuses primarily on the technical architecture. Fig. 6 shows the resulting technical architecture. This technology architecture can be found in the third row of the Zachman framework in Table 3. The seven software components identified above and their internal services are illustrated here showing how the services of the seven components come together to make an overall system architecture.

F. STEP 6 : IMPLEMENTATION

This section presents the implementation of one possible instance of the proposed technical architecture and three use cases as example smart building applications. The use cases and the corresponding use case buildings (Living Labs) are as follows:

1) Use Case 1: A Digital Twin application for the Atlas campus building at Eindhoven University of Technology. Digital twin applications involves integrating sensor data (both historical and real-time) with a digital model of a building [92]–[95]. This task is not however achieved in the digital twin applications literature in a modular and reusable manner. This is explained by the complexity of handing the heterogeneous IoT systems as we described earlier. Majority of the demonstrations [96]–[98] are done using Revit and Dynamo Visual Programming Language for historical sensor data integration. The downside of using these commercial off the shelf software is that it is vendor specific, and therefore the re-usability and extensibility is limited. In contrast, we propose a vendor-agnostic method for sensor data integration with BIM [22] based on the proposed architecture. We use time series storage to record the historical sensor data. Metadata which describes BIM and BMS systems lies in the graph database. SPARQL queries executes against metadata graph provides necessary relationships (spaces, sensor ids) to link the digital BIM model with sensor data. This way we realize a modular and reusable approach for creating digital twins.
2) Use Case 2: Integration of real-time sensor data for the Atlas campus building at Eindhoven University of Technology. Other essential component for a digital twin is the real time data. For the demonstration purpose, realtime sensor data from multiple IoT sensors are used here such as IEQ (temperature, humidity, light, TVOC and CO_2) and real-time energy consumption data (power, current). This application is developed with the aim of integrating real-time IoT sensor data coming from heterogeneous sources and visualizing them in a web application, alongside the semantic information of the building and the IoT devices.
3) Use Case 3: Integration of a semantic building graph with a Grafana dashboard for Building 33 at Delft University of Technology. One of the primary objectives of this use case building is to provide historical and real-time access to BMS sensor data for the development of data-driven smart building applications. However, the sensor data and metadata are not understood properly by users by default because of ambiguous semantics used by the vendor and are, therefore, difficult to use for data-driven applications. To ensure the understanding of the sensor data, contextual data was first standardised using smart building ontologies. This contextual data enables filtering the sensor data of the equipment, sensors, and points of interest. To improve the usability, we presented the integration using Grafana dashboards where semantic and timeseries data can be queried in combination.

1) Existing business applications (legacy systems)

We rely on the BMS, BIM and IoT systems for the implementations. The current purpose of the interaction with the BMS is only to extract historical data. Existing BIM models, developed using Revit, are used to create Digital Twin applications. Other source systems include IoT devices which provide real-time sensor data. To integrate these existing systems, we use multiple drivers, which are described in the integration software section.

2) New microservices-based applications

New applications reside primarily in the integration software and new business applications categories. At this stage our services include IoT drivers, BMS drivers, and file converters and ETL tools that provides data to the system. Other planned services include data-driven applications like FDD and MPC.

3) Databases

For the implementation, we rely on the four types of databases described in Section 5. MongoDB time series collection is used to store time series data, whereas MongoDB document collection is used for storing other data regarding users and projects. GraphDB database stores the semantic graphs describing metadata of the buildings. As an object storage we use Minio to store IFC files, converted XKT files, and large timeseries data collections, etc. All databases are installed in the Docker environment.

An example of the physical data model of a MongoDB document stored in the time series data collection is shown in Listing 2:

Listing 2. time series data stored in MongoDB time series collection

```
{
    "_id": ObjectId("6124567890abcdef12345678"),
    "sensor_id": "WS001",
    "timestamp": ISODate("2023-07-28T12:00:00Z"),
```


This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

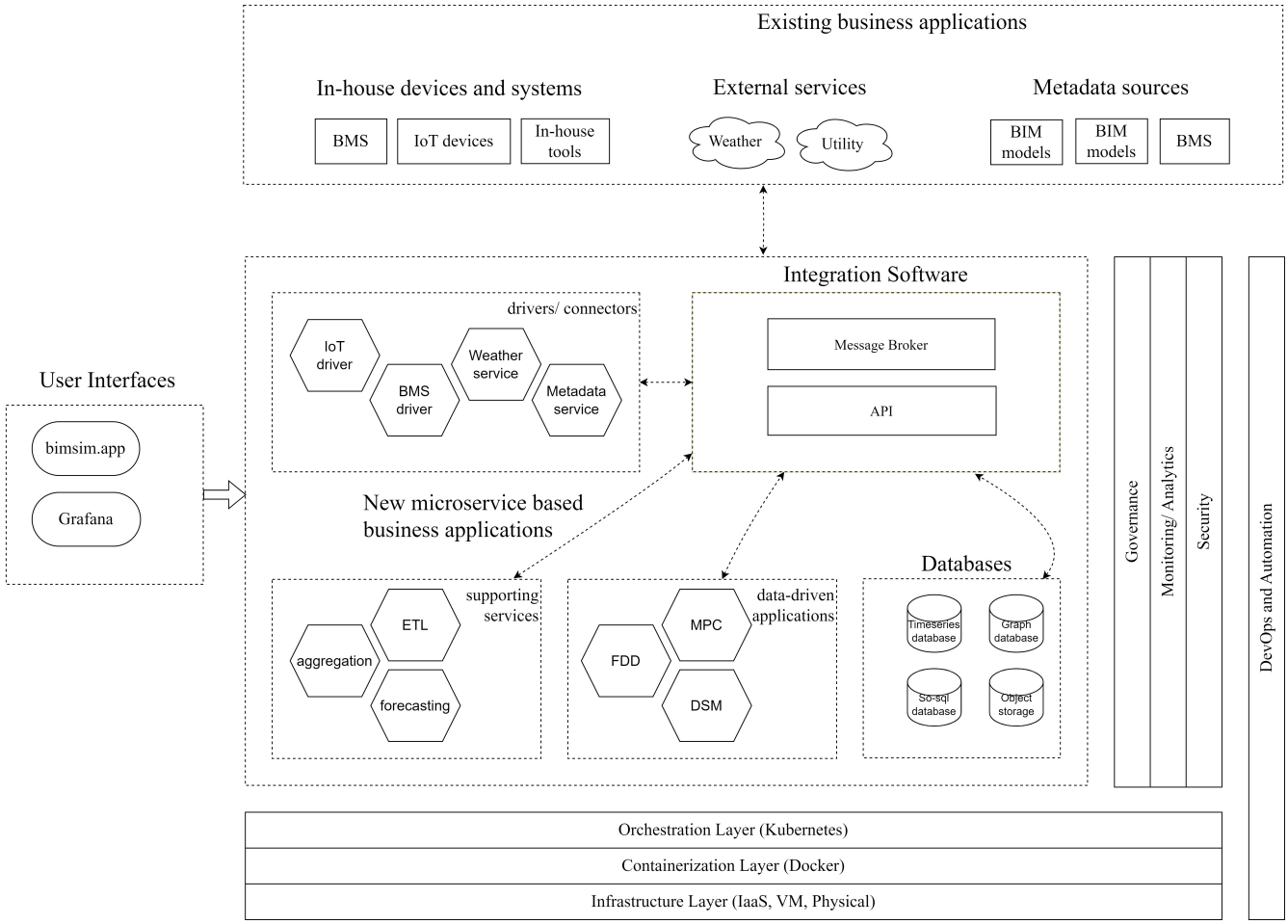

FIGURE 6. Proposed service-oriented software architecture for a data-driven smart building.

```
    "value": 28.5
}
```

The physical data model of the project data stored in MongoDB collection is shown in Listing 3:

Listing 3. Project data stored in MongoDB collection

```
{
    "_id": ObjectId("64dcfe2a7de01fc9ada366e6"),
    "title":"TU Eindhoven",
    "description":"Atlas building",
    "xktFiles":["atlas.xkt"],
    "rdfFiles":[""atlas.ttl"],
    "queries":[],
    "__v":1
}
```

A part of the semantic building graph stored in the GraphDB database is shown in Listing 4. The same graph is visually represented in Fig. 7.

Listing 4. A part of the semantic graph stored in RDF format in GraphDB graph database

```
inst:08BEAC0A1C04_Edimax a brick:Sensor ;
rdfs:label "EDIMAX air quality sensor";
  brick:hasPoint
    inst:08BEAC0A1C04_Edimax_co2,
    inst:08BEAC0A1C04_Edimax_h ;
    brick:hasLocation inst:space_892 ;
ref:hasExternalReference [
        a ref:time seriesReference ;
        ref:hastime seriesId "edimax/08BEAC0A1C04/pub" ;
        ref:storedAt :timeseries-database ;
    ] .

inst:11NR008LT-001PIRTM_Sensor a brick:Occupancy_Sensor ;
    rdfs:label "PRESENCE_8_128"^^xsd:string ;
    brick:hasLocation inst:space_892 ;
ref:hasExternalReference [
        a ref:timeseriesReference ;
        ref:hastime seriesId "11NR008LT-001PIRTM" ;
        ref:storedAt :time series-database ;
    ] .

inst:space_892
        a bot:Space, brick:Room ;
        bot:hasGuid
 "1456e859-2e9e-4e70-9341-f073adb2dbb0"^^xsd:string;
        props:hasCompressedGuid
 "0KLkXPBfvES9D1y7Ejijkm"^^xsd:string .
```

4) Integration software

BMS integration

At this stage, we use historical data from the BMS for Use Case 1 and Use Case 3. An ETL process is used to transform the incoming data into the MongoDB schema defined in the logical data model. Protocol-specific drivers will be implemented in future developments to establish the

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

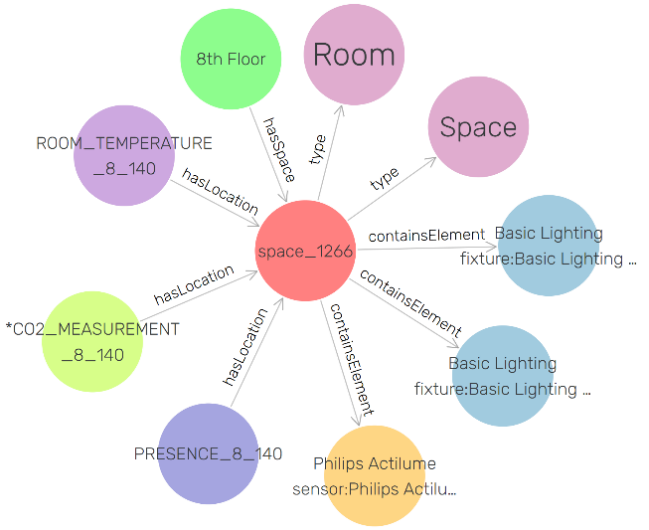

FIGURE 7. Visual representation of the Semantic graph combining multiple source systems using GraphDB interface.

bidirectional communication required for applications like FDD and MPC.

IoT integration

IoT integration is done in Use Case 2, the TU/e Atlas building. To demonstrate the IoT integration, we used three types of IoT devices. The first device is an air quality sensor, the data from which is accessed in real-time by querying the manufacturer's API. The IoT driver for this device fetches sensor data every 10 seconds from the IoT platform and publishes them to the MQTT broker. The second device is a smart socket. It uses a proprietary protocol, and therefore, the IoT driver for this device adapts an already available connector [99] to forward these messages to the MQTT broker. The third IoT device is custom-made by the authors using an ESP32 development board. This device communicates its readings to the MQTT broker directly. This way, all real-time data becomes accessible via the single MQTT message broker. Each IoT device ID (optionally MAC address, if available) is included in the MQTT topic, and the sensor readings are included in the MQTT payload. The payload of each IoT device is processed using another microservice as previously demonstrated in Fig. 5 to transform the payloads into a uniform structure. The intended logical data model for this service is shown in Listing 5. This data model is compatible with many IoT payloads. This unified data is also recorded in a time series database for later usage.

```
{
  topic: 'esp32/083AF266DD84/pub',
  keys: [ 'co2', 'tvoc',
  'te', 'rh', 'lux', 'vbat' ],
  values: [ 872, 71, 27.3233,
  56.39648, 586.6666, 4.1569 ],
  timestamp: 1662298546131
}
```

Listing 5. Logical data model of the processes messages

BIM integration

BIM integration is used in Use Case 1 in the Atlas building. For BIM integration, we rely on the xeokit SDK. This implementation builds on previous work describing the implementation of a web application to visualize a web-based BIM model using xeokit [22]. In this case, another driver is required to convert IFC files to a web-compatible XKT format. This conversion is achieved using the conversion process elaborated in ifc2xkt [89]. The converted files are stored in an object storage which can be accessed via the API.

Metadata integration

All use cases described above rely on the metadata integration service. The goal of the metadata graph is to standardise the semantics and create a link between different data sources. The three most critical data sources are the BMS, IoT and the BIM model, as they contain a large amount of metadata that enables contextual representations in data-driven applications. In this regard, two types of metadata drivers were implemented, one for the IFCto_RDF conversion and another for generating a metadata schema based on BMS and IoT sensors.

Creating a metadata schema from an IFC model requires a purpose-built converter to understand the available resources and map them to a desired ontology. This study achieves this through the implementation of an IFC to RDF driver. In the IFC-to-RDF conversion, the semantic model of the building is generated based on the IFC file using an IFC-to-RDF converter [90]. It transforms the IFC statements into RDF statements conforming to the PROPS, BOT and MEP ontologies.

Brick and SSN ontologies can be used to describe the BMS and IoT domains respectively, which requires a sensor metadata driver. First, semantics are extracted from the metadata sources like data dictionaries and device specifications in BMS to create a metadata schema [100]. These can be parsed using tools like Brick-builder tool [91] to create an RDF graph. Similar to BMS device IDs, IoT data streams are identified by their *topics*. There is no universal way to define a topic in MQTT; it uses best practices using a combination of the location and MAC address of the device. Therefore, topics can be considered an identifier for real-time data and can be included in the semantic graph. These identifiers are included using a `ref:hastime seriesId` relationship. These graphs are stored in the GraphDB graph database and queried using SPARQL. Discovering the sensors and observations using their sensor type, location, or other relationships can be done by executing SPARQL queries on the graph database via the API. As an example graph, a part of such metadata schema with building and sensor semantics of the Atlas Living Lab is shown in Listing. 4, and a graphical version is shown in Fig. 7.

API

The purpose of the REST API is to facilitate the communication between the smart building applications and the microservices in the architecture. All three use cases use the same API. The objective is to keep this API more general and applicable to any data-driven building, not specific to one use

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

case.

The API is implemented in Node.js using Nest.js framework, and sample endpoints in openAPI specification are shown in Fig. 8. It allows applications to interact with various services and execute Create-Read-Update-Delete (CRUD) operations against the available four databases. Time series data can be accessed via the API by querying with the relevant unique sensor ID. The API also provides endpoints to query the graphs in the RDF database, the project data stored in the document database, and the files stored in the Minio storage.

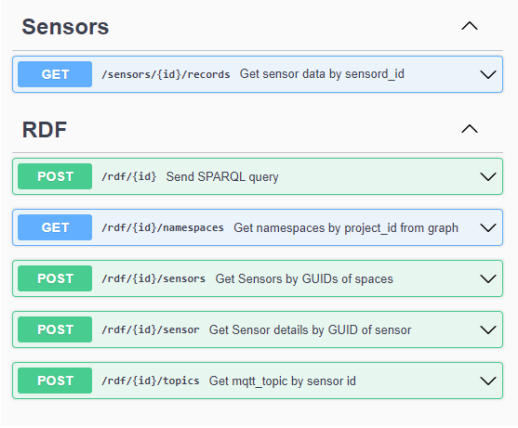

FIGURE 8. Sample endpoints for interacting with the various services, sensor data and metadata schema of the building.

The following section discusses an example of interaction with the API.

The first instance includes querying for sensor metadata using a known space identifier (e.g., IFC GUID of a space). For this, we need to execute a SPARQL query as shown in Listing 6, which enables getting the sensors in a particular space. It returns the identifiers of the sensors contained in a given space using *brick:hasPoint* relationship. These are the unique sensor identifiers used to record historical sensor data in the time series database.

```
SELECT ?space ?location ?sensor_id ?sensor_type ?topic
where {
          VALUES ?value {
            ${spaces.map(space => `'${space}'`).join(' ')}
          }
          ?location props:hasCompressedGuid ?value .
          ?location props:hasCompressedGuid ?space .
          ?item brick:hasLocation ?location .
          ?item brick:hasPoint ?sensor_id .
          ?item a ?sensor_type .
          OPTIONAL{
          ?item ref:hasExternalReference ?arr .
          ?arr ref:hastime seriesId ?topic .
          }
      }
```

Listing 6. SPARQL query to get all sensors in a space

Now, the execution of these kind of queries are encapsulated in the API endpoints to improve re-usability. For example, Listing 7 shows an API request that takes the GUID of the space and returns the sensors in it.

Listing 7. Get sensors by GUID of space

```
Method          : POST
URL             : /rdf/{id}/sensors
Request body    : {
  "spaces": [
    "0KLkXPBfvES9D1y7EjijkE"
  ]}
Response        :
 [
  {
    "space": {
      "type": "literal",
      "value": "0KLkXPBfvES9D1y7EjijkE"
    },
    "location": {
      "type": "uri",
      "value": "inst:space_1266"
    },
    "sensor_id": {
      "type": "uri",
      "value": "inst:083AF266DD84_ESP32_co2"
    },
    "sensor_type": {
      "type": "uri",
      "value": "https://brickschema.org/schema/Brick#Sensor"
    },
    "topic": {
      "type": "literal",
      "value": "esp32/083AFXXXXXXX/pub"
    }
  },
  .
  .
  ]
```

Further, these sensor IDs can be used to retrieve relevant sensor data from the time series storage as shown in Listing 8.

Listing 8. Get sensor records by sensor id

```
Method          : GET
URL             : /sensors/{id}/records
Query parameters: sensor_id, start date, end date
Response        : {
  "sensor_id": "11NR00STE-001TRL",
  "start": "2021-01-01T10:06:03.837Z",
  "end": "2022-01-01T10:06:03.837Z",
  "result": [
    {
      "timestamp": "2021-04-30T20:02:00.000Z",
      "value": 22
    },
    {
      "timestamp": "2021-04-30T11:02:00.000Z",
      "value": 21.9
    },
    {
      "timestamp": "2021-04-30T08:02:00.000Z",
      "value": 22.4
    },
    .
    .
    ]
}
```

5) Infrastructure services

We use Docker as the containerization environment, as newer applications, such as IoT drivers, API, and databases, run on Docker.

6) User interfaces

This section demonstrates the implementations of two web applications for Use Case 1 and 2, and a Grafana dashboard for Use Case 3. All applications are developed based on the proposed and implemented architecture.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

The first web application is developed for Use Case 1 in the Atlas building using the React front-end framework. The application provides the user to navigate through the BIM model and see the sensor data related to its spaces. The REST API connects the web application to the semantic graph, time series data (both historical and real-time) and BIM models. Figure 9 demonstrates historical sensor data linked to a BIM model and visualized in the web application [22]. Figure 9 also shows how historical data is retrieved for a CO_2 sensor installed inside a room on the 8th floor of the Atlas building.

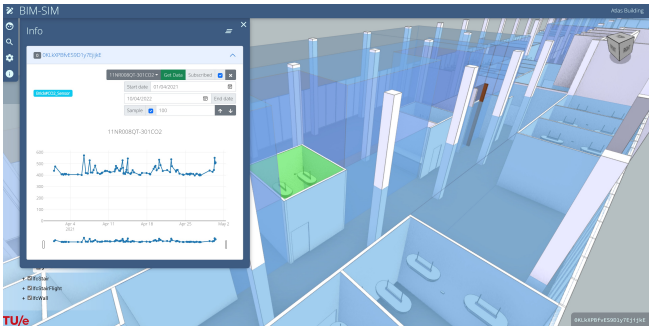

FIGURE 9. Integration of historical CO2 sensor data of the selected space during April 2021 and May 2021 [22]

The second web application is developed for Use Case 2 based on the implemented architecture and is shown in Fig. 10. This web application was also developed using the React front-end framework. The application allows the user to interact with the metadata graph of the Atlas building to explore its IoT devices and then get their real-time data [87]. This application relies on the IoT drivers we implemented to unify the IoT payloads from three heterogeneous sources.

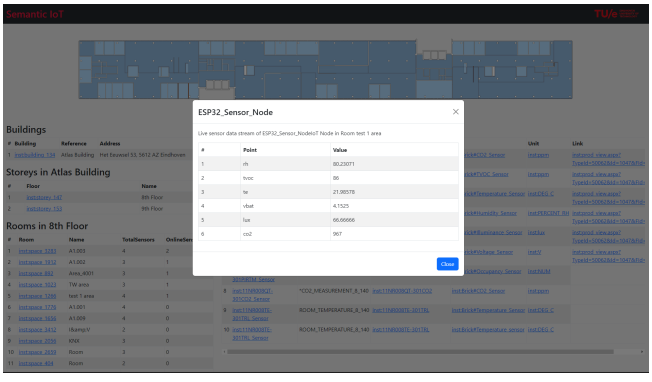

FIGURE 10. Integration of realtime sensor data using metadata graph [87]

The third application is developed as a data exploratory and visualisation tool for BMS data in Use Case 3. We developed a metadata schema for the BMS and integrated it with the time series database and a Grafana dashboard using the API, providing the ability to query time series data efficiently. That includes the ability to query data by equipment or point types and gain initial insights about the data through charts. Users can query the BMS points of interest by selecting the equipment type from the drop-down list in Figure 11. This list contains the equipment types from the BMS, annotated according to the Brick ontology.

Figure 11 shows an example of an Air Handling Unit (AHU) selected as *brick:Equipment*. Then, a SPARQL query requests the unique identifiers of the data points related to this *brick:AHU* in the metadata schema. The user can select one or more points from the resulting point list to explore their time series data. Overall, the architecture enables efficient means to understand and explore the large amounts of time series data available from the building.

FIGURE 11. BMS time series data exploration using metadata schema and Grafana dashboards [100]

V. CONCLUSION AND FUTURE WORK

A. CONCLUSION

Buildings have an abundance of operational and contextual data. Smart building applications are on the rise with the increasing attention to operational objectives related to energy efficiency, energy flexibility, IEQ, occupant comfort and well-being. For these applications, data must be linked, and their underlying systems must be integrated to exchange information. However, current system architectures are inadequate for smart building use cases because 1) they lack the integration of best practices such as architecture frameworks and matured architecture patterns, 2) they have not integrated semantic technologies as an architectural component and have showcased limited capabilities using ad-hoc tools, 3) the available system architectures are largely conceptual and practical implementations are rare. This study proposes a microservices-based architecture with reusable, modular and scalable components to overcome these challenges. We further implement three smart building applications atop the proposed architecture, proving its reusability and modularity.

Most existing applications show limitations, as they often rely on proprietary data models and non-scalable solutions. In contrast, our approach shows a significant improvement in scalability, modularity and reusability due to the utilised MACH principles and ontology-based extensible metadata schemes. For example, the presented BIM-IoT web application for the Atlas use case building can be easily extended and used in another building. Having created a metadata schema

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

and BIM model of the new building, it can use the same application logic, and, therefore, no additional effort is required to configure the application. This is also applicable to the other two applications demonstrated in the implementation. Other applications, such as FDD and MPC, are currently under implementation as new microservices.

Implementating an instance of the proposed reference architecture and the three smart building applications reveal that the proposed architecture is suitable as a blueprint to create similar system architecture for smart buildings. We demonstrate how integration software and an API can integrate the existing heterogeneous information sources with new microservices-based applications. Furthermore, we show how different data types should be maintained using different databases. The original data types are retained in their optimum environments by choosing appropriate databases. In contrast to custom data models, we opted for metadata models based on open standards and well-established ontologies such as Brick, SSN, BOT, etc. This way, the data model remains flexible and extensible.

Besides other researchers, this article is relevant for solution architects who want to understand the building domain, as well as business executives and owners in the building domain looking to build solution architectures for buildings. This type of architecture is also directly useful for service providers who develop and provide software for smart buildings. For other service providers, such as energy companies or algorithm providers who wish to interact with the building domain, this software architecture provides a standard API, making the data and systems accessible and avoiding any ambiguities when implementing new smart building applications.

B. FUTURE WORK

Future work includes implementing data-driven applications such as MPC and FDD as new microservice-based applications. These applications can be developed and deployed individually in containerised environments, relying on the core of the system architecture, the API and the message broker for communication. To that end, additional services for data analytics, Machine Learning, and tools required for exploratory data analysis will be implemented as new services.

Utilising semantic technologies as an integral part of the architecture is an essential component. However, generating the metadata schema from available metadata sources can be quite a laborious process if the metadata is not well-documented or not available in a structured format such as CSV. The efficient generation of metadata graphs itself is an emerging research area, which tries to automate the process [14], [101], and improve the semantic sufficiency of the generated graph for a particular applications [102]. When matured, these services should be integrated with the proposed architecture.

Although the three implemented smart building applications do not use bidirectional communication with the building controllers, future developments like FDD and MPC rely on this active bidirectional communication. The idea is to control the devices and controllers using intelligent algorithms, that send optimal set points and control commands to the device APIs via relevant drivers. Using an event-driven architecture [103] that uses events to share information between decoupled services is most suitable for these intelligent control applications. The current implementation of the architecture relies on a REST API and a message broker for sharing information between systems. An event-driven service component will also be added to the architecture in the future. This addition is valuable for continuously monitoring the devices and triggering the relevant services that rely on those events.

It is also essential to investigate how data could be categorised into internal, shared and open data since the privacy and security aspects of each can be different. A standardised methodology for secure and ethical access and use of data (collection, management and use) and for controlling the devices is also under investigation [19].

ACKNOWLEDGEMENTS

the Brains for Buildings project received funding from the Dutch Ministry of Economic Affairs and Climate Policy and Ministry of the Interior and Kingdom Relations under the MOOI program.

REFERENCES

[1] P. Arjunan, M. Srivastava, A. Singh, and P. Singh, "OpenBAN: An Open Building ANalytics Middleware for Smart Buildings," in *Proceedings of the 12th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*. ACM, 2015, pp. 70–79. [Online]. Available: http://www.bacnet.orghttp://eudl.eu/doi/10.4108/eai.22-7-2015.2260256

[2] B. Balaji, A. Bhattacharya, G. Fierro, J. Gao, J. Gluck, D. Hong, A. Johansen, J. Koh, J. Ploennigs, Y. Agarwal, M. Bergés, D. Culler, R. K. Gupta, M. B. Kjærgaard, M. Srivastava, and K. Whitehouse, "Brick : Metadata schema for portable smart building applications," *Applied Energy*, vol. 226, pp. 1273–1292, sep 2018. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0306261918302162

[3] X. Gao, P. Pishdad-Bozorgi, D. R. Shelden, and S. Tang, "Internet of Things Enabled Data Acquisition Framework for Smart Building Applications," *Journal of Construction Engineering and Management*, vol. 147, no. 2, p. 04020169, feb 2021. [Online]. Available: https://ascelibrary.org/doi/10.1061/{%}28ASCE{%}29CO.1943-7862.0001983

[4] R. Cox, S. Walker, J. van der Velden, P. Nguyen, and W. Zeiler, "Flattening the Electricity Demand Profile of Office Buildings for Future-Proof Smart Grids," *Energies*, vol. 13, no. 9, p. 2357, may 2020. [Online]. Available: https://www.mdpi.com/1996-1073/13/9/2357

[5] A. Krishnan Prakash, K. Zhang, P. Gupta, D. Blum, M. Marshall, G. Fierro, P. Alstone, J. Zoellick, R. Brown, and M. Pritoni, "Solar+ Optimizer: A Model Predictive Control Optimization Platform for Grid Responsive Building Microgrids," *Energies*, vol. 13, no. 12, p. 3093, jun 2020. [Online]. Available: https://www.mdpi.com/1996-1073/13/12/3093

[6] S. Lazarova-Molnar, H. R. Shaker, N. Mohamed, and B. N. Jorgensen, "Fault detection and diagnosis for smart buildings: State of the art, trends and challenges," in *2016 3rd MEC International Conference on Big Data and Smart City (ICBDSC)*. IEEE, mar 2016, pp. 1–7. [Online]. Available: http://ieeexplore.ieee.org/document/7460392/

[7] M. S. Mirnaghi and F. Haghighat, "Fault detection and diagnosis of large-scale HVAC systems in buildings using data-driven methods: A comprehensive review," *Energy and Buildings*, vol. 229, p. 110492, dec 2020. [Online]. Available: https://doi.org/10.1016/j.enbuild.2020.110492

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

[8] M. Bird, C. Daveau, E. O'Dwyer, S. Acha, and N. Shah, "Real-world implementation and cost of a cloud-based MPC retrofit for HVAC control systems in commercial buildings," *Energy and Buildings*, vol. 270, p. 112269, sep 2022. [Online]. Available: https://doi.org/10.1016/j.enbuild.2022.112269

[9] D. Blum, Z. Wang, C. Weyandt, D. Kim, M. Wetter, T. Hong, and M. A. Piette, "Field demonstration and implementation analysis of model predictive control in an office HVAC system," *Applied Energy*, vol. 318, p. 119104, jul 2022. [Online]. Available: https://doi.org/10.1016/j.apenergy.2022.119104https://linkinghub.elsevier.com/retrieve/pii/S0306261922004895

[10] Z. Chevallier, B. Finance, and B. C. Boulakia, "A reference architecture for smart building digital twin," *CEUR Workshop Proceedings*, vol. 2615, pp. 1–12, 2020.

[11] G. B. Ozturk, "Digital Twin Research in the AECO-FM Industry," *Journal of Building Engineering*, vol. 40, p. 102730, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S235271022100588X

[12] S. Aheleroff, X. Xu, R. Y. Zhong, and Y. Lu, "Digital Twin as a Service (DTaaS) in Industry 4.0: An Architecture Reference Model," *Advanced Engineering Informatics*, vol. 47, p. 101225, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1474034620301944

[13] P. Pauwels, S. Zhang, and Y.-C. C. Lee, "Semantic web technologies in AEC industry: A literature overview," *Automation in Construction*, vol. 73, pp. 145–165, jan 2017. [Online]. Available: http://dx.doi.org/10.1016/j.autcon.2016.10.003https://linkinghub.elsevier.com/retrieve/pii/S0926580516302928

[14] A. A. Bhattacharya, D. Hong, D. Culler, J. Ortiz, K. Whitehouse, and E. Wu, "Automated metadata construction to support portable building applications," in *Proceedings of the 2nd ACM International Conference on Embedded Systems for Energy-Efficient Built Environments*. New York, NY, USA: ACM, nov 2015, pp. 3–12. [Online]. Available: https://dl.acm.org/doi/10.1145/2821650.2821667

[15] Y. Agarwal, R. Gupta, D. Komaki, and T. Weng, "BuildingDepot," in *Proceedings of the Fourth ACM Workshop on Embedded Sensing Systems for Energy-Efficiency in Buildings*. New York, NY, USA: ACM, nov 2012, pp. 64–71. [Online]. Available: https://dl.acm.org/doi/10.1145/2422531.2422545

[16] M. Pau, P. Kapsalis, Z. Pan, G. Korbakis, D. Pellegrino, and A. Monti, "MATRYCS—A Big Data Architecture for Advanced Services in the Building Domain," *Energies*, vol. 15, no. 7, p. 2568, apr 2022. [Online]. Available: https://doi.org/10.3390/en15072568

[17] C. Fernando, *Solution Architecture Patterns for Enterprise*. Berkeley, CA: Apress, 2023. [Online]. Available: https://doi.org/10.1007/978-1-4842-8948-8

[18] Z. JA, " A Framework for Information Systems Architecture," *IBM Systems Journal*, vol. 26, no. 3, pp. 267–292, 1987.

[19] Work Packages – Brains4buildings. https://brains4buildings.org/work-packages/. Accessed: 2023-08-07.

[20] M. Richards, *Microservices vs. service-oriented architecture*. Berlin, Heidelberg: O'Reilly Media, 2016. [Online]. Available: https://www.oreilly.com/library/view/microservices-vs-service-oriented/9781491975657/

[21] Enterprise MACHified | MACH Alliance. https://machalliance.org/. Accessed: 2023-08-07.

[22] L. Chamari, E. Petrova, and P. Pauwels, "A web-based approach to BMS, BIM and IoT integration: a case study," in *REHVA 14th HVAC World Congress*, 2022, pp. 1–8. [Online]. Available: https://proceedings.open.tudelft.nl/clima2022/article/view/228

[23] N. Medvidovic and R. N. Taylor, "Software architecture," in *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering - Volume 2*, ser. ICSE '10. New York, NY, USA: ACM, may 2010, pp. 471–472. [Online]. Available: https://doi.org/10.1145/1810295.1810435

[24] M. Guessi, V. V. G. Neto, T. Bianchi, K. R. Felizardo, F. Oquendo, and E. Y. Nakagawa, "A systematic literature review on the description of software architectures for systems of systems," in *Proceedings of the 30th Annual ACM Symposium on Applied Computing*, ser. SAC '15. New York, NY, USA: ACM, apr 2015, pp. 1433–1440. [Online]. Available: https://doi.org/10.1145/2695664.2695795

[25] M. M. Lankhorst, *Enterprise architecture at work: Modelling, communication and analysis, fourth edition*, 2017. [Online]. Available: https://link.springer.com/book/10.1007/978-3-662-53933-0

[26] Industry IoT Consortium, "The Industrial Internet Reference Architecture," Tech. Rep., 2022. [Online]. Available: https://www.iiconsortium.org/IIRA/

[27] T. O. Group. (2011) The Open Group Architectural Framework (TOGAF) Version 9.1. [Online]. Available: http://www.opengroup.org/togaf/

[28] K. Smolander, "What is included in software architecture? A case study in three software organizations," in *Proceedings Ninth Annual IEEE International Conference and Workshop on the Engineering of Computer-Based Systems*. IEEE Comput. Soc, 2002, pp. 131–138. [Online]. Available: http://ieeexplore.ieee.org/document/999831/

[29] J. a. Zachman, "The Zachman Framework For Enterprise Architecture, Primer for Enterprise Engineering and Manufacturing," *CA Magazine*, vol. 128, no. 9, p. 15, 2003. [Online]. Available: http://proquest.umi.com/pqdweb?did=9075313{&}Fmt=7{&}clientId=65345{&}RQT=309{&}VName=PQD

[30] T. Ylimäki and V. Halttunen, "Method engineering in practice: A case of applying the Zachman framework in the context of small enterprise architecture oriented projects," *Information, Knowledge, Systems Management*, vol. 5, no. 3, pp. 189–209, 2005. [Online]. Available: https://dl.acm.org/doi/10.5555/2656974.2656977

[31] A. Radwan and M. Aarabi, "Study of Implementing Zachman Framework for Modeling Information Systems for Manufacturing Enterprises Aggregate Planning," in *Proceedings of the 2011 International Conference on Industrial Engineering and Operations Management*, 2011, pp. 9–14. [Online]. Available: https://ieomsociety.org/ieom2011/pdfs/IEOM003.pdf

[32] H. Tannady, J. F. Andry, B. G. Sudarsono, and Y. Krishartanto, "Enterprise Architecture Using Zachman Framework at Paint Manufacturing Company," *Technology Reports of Kansai University*, vol. 62, no. 4, pp. 1869–1883, 2020.

[33] J. M. Nogueira, D. Romero, J. Espadas, and A. Molina, "Leveraging the Zachman framework implementation using action - research methodology - a case study: Aligning the enterprise architecture and the business goals," *Enterprise Information Systems*, vol. 7, no. 1, pp. 100–132, 2013.

[34] S. Bondar, J. C. Hsu, A. Pfouga, and J. Stjepandić, "Agile digital transformation of System-of-Systems architecture models using Zachman framework," *Journal of Industrial Information Integration*, vol. 7, pp. 33–43, 2017.

[35] R. Alm and M. Wißotzki, "TOGAF adaption for small and medium enterprises," *Lecture Notes in Business Information Processing*, vol. 160, pp. 112–123, 2013.

[36] S. Kotusev, "TOGAF-based enterprise architecture practice: An exploratory case study," *Communications of the Association for Information Systems*, vol. 43, no. 1, pp. 321–359, 2018.

[37] B. Anthony, S. A. Petersen, D. Ahlers, J. Krogstie, and K. Livik, "Big data-oriented energy prosumption service in smart community districts: a multi-case study perspective," *Energy Informatics*, vol. 2, no. 1, p. 36, dec 2019. [Online]. Available: https://doi.org/10.1186/s42162-019-0101-3

[38] W. Engelsman and R. Wieringa, "Goal-oriented requirements engineering and enterprise architecture: Two case studies and some lessons learned," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 7195 LNCS, no. iv, pp. 306–320, 2012.

[39] OMG. (2017) About the Unified Modeling Language Specification Version 2.5.1. https://www.omg.org/spec/UML. Accessed: 2023-07-03.

[40] T. O. Group. (2017) ArchiMate® 3.2 Specification. [Online]. Available: https://pubs.opengroup.org/architecture/archimate3-doc/

[41] M. M. Lankhorst, L. van der Torre, H. A. E. Proper, F. Arbab, S. J. B. A. Hoppenbrouwers, and M. W. A. Steen, *Viewpoints and Visualisation*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2017, pp. 171–214. [Online]. Available: https://doi.org/10.1007/978-3-662-53933-0{_}8

[42] I. Malavolta, P. Lago, H. Muccini, P. Pelliccione, and A. Tang, "What industry needs from architectural languages: A survey," *IEEE Transactions on Software Engineering*, vol. 39, no. 6, pp. 869–891, 2013.

[43] Philippe Kruchten, "The 4+1 View Model of architecture," *IEEE Software*, vol. 12, no. 6, pp. 42–50, 1995. [Online]. Available: http://ieeexplore.ieee.org/document/469759/

[44] M. Richards, *Software Architecture Patterns, 2nd Edition*. Sebastopol, CA: O'Reilly Media, Inc., 2022. [Online]. Available: https://www.oreilly.com/library/view/software-architecture-patterns/9781491971437/colophon01.html

[45] Atlassian. Microservices vs. monolithic architecture. https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith. Accessed: 2023-08-07.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

[46] A. Kharenko. Monolithic vs. Microservices Architecture. https://articles.microservices.com/monolithic-vs-microservices-architecture-5c4848858f59. Access date: 2023-08-07.

[47] AWS. What's the Difference Between Monolithic and Microservices Architecture? https://aws.amazon.com/compare/the-difference-between-monolithic-and-microservices-architecture/. Accessed: 2023-08-07.

[48] D. Shadija, M. Rezai, and R. Hill, "Towards an understanding of microservices," in *2017 23rd International Conference on Automation and Computing (ICAC)*, 2017, pp. 1–6.

[49] H. Calderón-Gómez, L. Mendoza-Pittí, M. Vargas-Lombardo, J. M. Gómez-Pulido, D. Rodríguez-Puyol, G. Sención, and M.-L. Polo-Luque, "Evaluating Service-Oriented and Microservice Architecture Patterns to Deploy eHealth Applications in Cloud Computing Environment," *Applied Sciences*, vol. 11, no. 10, p. 4350, may 2021. [Online]. Available: https://www.mdpi.com/2076-3417/11/10/4350

[50] M. P. Papazoglou and W.-J. van den Heuve, "Service oriented architectures: approaches, technologies and research issues," *The VLDB Journal*, vol. 16, pp. 389–415, 2007.

[51] R. Perrey and M. Lycett, "Service-oriented architecture," in *2003 Symposium on Applications and the Internet Workshops, 2003. Proceedings.*, 2003, pp. 116–119.

[52] I. Jerstad, S. Dustdar, and D. V. Thanh, "A service oriented architecture framework for collaborative services," in *14th IEEE International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprise (WETICE'05)*, 2005, pp. 121–125.

[53] M. Rychlý. and P. Weiss., "Modelinf of Service Oriented Architecture - From Business Process to Service Realisation," in *Proceedings of the Third International Conference on Evaluation of Novel Approaches to Software Engineering - ENASE*, INSTICC. SciTePress, 2008, pp. 140–146.

[54] Min Luo, B. Goldshlager, and Liang-Jie Zhang, "Designing and implementing Enterprise Service Bus (ESB) and SOA solutions," in *2005 IEEE International Conference on Services Computing (SCC'05) Vol-1*, vol. 2. IEEE, 2005, p. xiv vol.2. [Online]. Available: http://ieeexplore.ieee.org/document/1524413/

[55] T. M. Żmuda, M. Psiuk, D. Radziszowski, T. Szyd, R. Szymacha, K. Zielinski, and Daniel, "ESB-Modern SOA Infrastructure," 2010. [Online]. Available: https://api.semanticscholar.org/CorpusID:53669102

[56] S. Katipamula and M. R. Brambley, "Review Article: Methods for Fault Detection, Diagnostics, and Prognostics for Building Systems—A Review, Part I," *HVAC&R Research*, vol. 11, no. 1, pp. 3–25, 2005. [Online]. Available: https://www.tandfonline.com/doi/abs/10.1080/10789669.2005.10391123

[57] P. Delgoshaei and M. A. Austin, "Framework for Knowledge-Based Fault Detection and Diagnostics in Multi-Domain Systems: Application to Heating Ventilation and Air Conditioning Systems," *International Journal on Advances in Intelligent Systems,*, vol. 10, no. 3, pp. 393–409, 2017.

[58] K. Zhang, A. Prakash, L. Paul, D. Blum, P. Alstone, J. Zoellick, R. Brown, and M. Pritoni, "Model predictive control for demand flexibility: Real-world operation of a commercial building with photovoltaic and battery systems," *Advances in Applied Energy*, vol. 7, no. July, p. 100099, 2022. [Online]. Available: https://doi.org/10.1016/j.adapen.2022.100099

[59] K. Bao, I. Mauser, S. Kochanneck, H. Xu, and H. Schmeck, "A Microservice Architecture for the Intranet of Things and Energy in Smart Buildings," in *Proceedings of the 1st International Workshop on Mashups of Things and APIs*. New York, NY, USA: ACM, dec 2016, pp. 1–6. [Online]. Available: https://dl.acm.org/doi/10.1145/3007203.3007215

[60] S. Dawson-Haggerty, A. Krioukov, J. Taneja, S. Karandikar, G. Fierro, N. Kitaev, and D. Culler, "BOSS: Building operating system services," *Proceedings of the 10th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2013*, pp. 443–457, 2013.

[61] M. Jahn, D. Berichter, and M. Jarke, "Turning Smart Buildings into Innovation Environments," Ph.D. dissertation, 2016. [Online]. Available: https://publications.rwth-aachen.de/record/682274/files/682274.pdf

[62] E. Patti, A. Acquaviva, M. Jahn, F. Pramudianto, R. Tomasi, D. Rabourdin, J. Virgone, and E. Macii, "Event-Driven User-Centric Middleware for Energy-Efficient Buildings and Public Spaces," *IEEE Systems Journal*, vol. 10, no. 3, pp. 1137–1146, sep 2016. [Online]. Available: https://ieeexplore.ieee.org/document/6740031

[63] T. R. Gruber, "A translation approach to portable ontology specifications," *Knowledge Acquisition*, vol. 5, no. 2, pp. 199–220, 1993. [Online]. Available: http://ac.els-cdn.com/S1042814383710083/1-s2.0-S1042814383710083-main.pdf?{_}tid=28c5c43c-df11-11e2-b19c-00000aacb35f{&}acdnat=1372327738{_}c3518b0aab22af436aef0c5e3e2dd48e

[64] K. Janowicz, M. H. Rasmussen, M. Lefrançois, G. F. Schneider, and P. Pauwels, "BOT: The building topology ontology of the W3C linked building data group," *Semantic Web*, vol. 12, no. 1, pp. 143–161, 2020.

[65] GitHub - w3c-lbd-cg/bot: Building Topology Ontology. https://github.com/w3c-lbd-cg/bot. Accessed: 2023-08-07.

[66] Semantic Sensor Network Ontology. https://www.w3.org/TR/vocab-ssn/. Accessed: 2023-08-07.

[67] K. Janowicz, A. Haller, S. J. Cox, D. Le Phuoc, and M. Lefrançois, "SOSA: A lightweight ontology for sensors, observations, samples, and actuators," *Journal of Web Semantics*, vol. 56, no. July 2019, pp. 1–10, may 2019. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S1570826818300295

[68] W. Terkaj, G. F. Schneider, and P. Pauwels, "Reusing domain ontologies in linked building data: The case of building automation and control," in *Proceedings of the Joint Ontology Workshops 2017 Episode 3: The Tyrolean Autumn of Ontology*, vol. 2050, 2017.

[69] Home - BrickSchema. https://brickschema.org/. Accessed: 2023-08-09.

[70] Home – Project Haystack. https://project-haystack.org/. Accessed: 2023-08-07.

[71] P. Delgoshaei, M. Heidarinejad, and M. A. Austin, "A Semantic Approach for Building System Operations: Knowledge Representation and Reasoning," *Sustainability*, vol. 14, no. 10, p. 5810, may 2022. [Online]. Available: https://www.mdpi.com/2071-1050/14/10/5810

[72] P. Delgoshaei, M. A. Austin, and D. Veronica, "Semantic Models and Rule-based Reasoning for Fault Detection and Diagnostics: Applications in Heating, Ventilating and Air Conditioning Systems," *The Twelfth International Conference on Systems*, no. c, pp. 48–53, 2017. [Online]. Available: https://pdfs.semanticscholar.org/b6aa/62285513182e28eef0d5977b524b108397cf.pdf

[73] Y.-Y. Zhang, Z.-Z. Hu, J.-R. Lin, and J.-P. Zhang, "Linking data model and formula to automate KPI calculation for building performance benchmarking," *Energy Reports*, vol. 7, pp. 1326–1337, nov 2021. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S2352484721001426

[74] H. Dibowski, J. Vass, O. Holub, and J. Rojicek, "Automatic setup of fault detection algorithms in building and home automation," in *2016 IEEE 21st International Conference on Emerging Technologies and Factory Automation (ETFA)*, vol. 2016-Novem. IEEE, sep 2016, pp. 1–6. [Online]. Available: http://ieeexplore.ieee.org/document/7733622/

[75] O. Topçu and H. Oğuztüzün, "High Level Architecture," in *Guide to Distributed Simulation with HLA*, 2017, no. June, pp. 29–78. [Online]. Available: https://aioti.eu/wp-content/uploads/2017/06/AIOTI-HLA-R3-June-2017.pdfhttp://link.springer.com/10.1007/978-3-319-61267-6{_}2

[76] E. Curry, A. Metzger, A. J. Berre, A. Monzón, and A. Boggio-Marzet, "A Reference Model for Big Data Technologies," in *The Elements of Big Data Value*. Springer International Publishing, 2021, pp. 127–151. [Online]. Available: https://link.springer.com/10.1007/978-3-030-68176-0{_}6

[77] M. Mazzara, I. Afanasyev, S. R. Sarangi, S. Distefano, V. Kumar, and M. Ahmad, "A reference architecture for smart and software-defined buildings," *Proceedings - 2019 IEEE International Conference on Smart Computing, SMARTCOMP 2019*, no. Section IV, pp. 167–172, 2019.

[78] M. Abu-Matar, "Towards a software defined reference architecture for smart city ecosystems," in *2016 IEEE International Smart Cities Conference (ISC2)*. IEEE, sep 2016, pp. 1–6. [Online]. Available: http://ieeexplore.ieee.org/document/7580807/

[79] M. Genkin and J. J. McArthur, "B-SMART: A Reference Architecture for Autonomic Smart Buildings." *IOP Conference Series: Earth and Environmental Science*, vol. 1101, no. 9, p. 092036, nov 2022. [Online]. Available: https://iopscience.iop.org/article/10.1088/1755-1315/1101/9/092036

[80] M. R. Bashir, A. Q. Gill, and G. Beydoun, "A Reference Architecture for IoT-Enabled Smart Buildings," *SN Computer Science*, vol. 3, no. 6, p. 493, sep 2022. [Online]. Available: https://doi.org/10.1007/s42979-022-01401-9

[81] M. R. Bashir, A. Q. Gill, G. Beydoun, and B. Mccusker, "Big data management and analytics metamodel for IoT-enabled smart buildings," *IEEE Access*, vol. 8, pp. 169 740–169 758, 2020. [Online]. Available: https://ieeexplore.ieee.org/document/9195814/

[82] Browse Azure Architectures - Azure Architecture Center | Microsoft Learn. https://learn.microsoft.com/en-

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2023.3325767

us/azure/architecture/browse/?azure%5C_categories=iot. Accessed: 2023-08-07.

[83] IoT | AWS Architecture Center. https://aws.amazon.com/architecture/iot/. Accessed: 2022-08-07.

[84] I. Roger Sessions ObjectWatch. (2007) A Comparison of the Top Four Enterprise-Architecture Methodologies. [Online]. Available: https://web.archive.org/web/20170310132123/https:/msdn.microsoft.com/en-us/library/bb466232.aspx{#}eacompar{_}topic3

[85] L. Chamari, P. Pauwels, E. Petrova, . Tu, E. Chochanova, R. Sebastian, N. Mutsaers, and S. Hopkins, "Deliverable D4.03 Study of data needs and requirements in smart buildings," Tech. Rep., 2021. [Online]. Available: www.brainsforbuildings.org

[86] NATS.io – Cloud Native, Open Source, High-performance Messaging. https://nats.io/. Accessed: 2023-08-10.

[87] L. Chamari, E. Petrova, and P. Pauwels, "Extensible real-time data acquisition and management for IoT enabled smart buildings," in *Proceedings of the 2023 European Conference on Computing in Construction and the 40th International CIB W78 Conference*, ser. Computing in Construction, vol. 4. Heraklion, Greece: European Council on Computing in Construction, jul 2023. [Online]. Available: https://ec-3.org/publications/conference/paper/?id=EC32023{_}300

[88] IFC.js. https://ifcjs.io/. Accessed: 2023-08-28.

[89] Home | xeokit-convert. https://xeokit.github.io/xeokit-convert/docs/. Accessed: 2022-08-07.

[90] GitHub - pipauwel/IFCtoRDF: IFCtoRDF is a set of reusable Java components that allows to parse IFC-SPF files and convert them into RDF graphs. https://github.com/pipauwel/IFCtoRDF. Accessed: 2023-08-07.

[91] GitHub - gtfierro/brick-builder. https://github.com/gtfierro/brick-builder. Accessed: 2023-08-07.

[92] S. van Gool, D. Yang, and P. Pauwels, "Integrating sensor- and building data flows: A case study of the IEQ of an office building in The Netherlands," *ECPPM 2021 – eWork and eBusiness in Architecture, Engineering and Construction*, pp. 328–333, 2021.

[93] G. Desogus, E. Quaquero, G. Rubiu, G. Gatto, and C. Perra, "Bim and iot sensors integration: A framework for consumption and indoor conditions data monitoring of existing buildings," *Sustainability (Switzerland)*, vol. 13, no. 8, 2021.

[94] N. Moretti, X. Xie, J. Merino, J. Brazauskas, and A. K. Parlikad, "An openBIM Approach to IoT Integration with Incomplete As-Built Data," *Applied Sciences*, vol. 10, no. 22, pp. 1–17, nov 2020. [Online]. Available: https://www.mdpi.com/2076-3417/10/22/8287

[95] H. Chen, L. Hou, G. K. Zhang, and S. Moon, "Development of BIM, IoT and AR/VR technologies for fire safety and upskilling," *Automation in Construction*, vol. 125, no. January, p. 103631, may 2021. [Online]. Available: https://doi.org/10.1016/j.autcon.2021.103631https://linkinghub.elsevier.com/retrieve/pii/S0926580521000820

[96] C. Quinn, A. Z. Shabestari, T. Misic, S. Gilani, M. Litoiu, and J. J. McArthur, "Building automation system - BIM integration using a linked data structure," *Automation in Construction*, vol. 118, no. June, p. 103257, 2020. [Online]. Available: https://doi.org/10.1016/j.autcon.2020.103257

[97] J. Teizer, M. Wolf, O. Golovina, M. Perschewski, M. Propach, M. Neges, and M. König, "Internet of Things (IoT) for integrating environmental and localization data in Building Information Modeling (BIM)," *ISARC 2017 - Proceedings of the 34th International Symposium on Automation and Robotics in Construction*, no. Isarc, pp. 603–609, jul 2017.

[98] P. Penna, G. L. Regis, A. Schweigkofler, C. Marcher, and D. Matt, "From Sensors to BIM: Monitoring Comfort Conditions of Social Housing with the KlimaKit Model," in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 2019, vol. 11792 LNCS, pp. 108–115. [Online]. Available: http://link.springer.com/10.1007/978-3-030-30949-7{_}12

[99] GitHub - TheAgentK/tuya-mqtt: Nodejs-Script to combine tuyaapi and openhab via mqtt. https://github.com/TheAgentK/tuya-mqtt. Accessed: 2023-08-07.

[100] L. Chamari, J. V. D. Weijden, L. Boonstra, and S. Hoekstra, "Metadata Schema Generation for Data-driven Smart Buildings," 2023.

[101] J. Koh, D. Hong, R. Gupta, K. Whitehouse, H. Wang, and Y. Agarwal, "Plaster," in *Proceedings of the 5th Conference on Systems for Built Environments*. New York, NY, USA: ACM, nov 2018, pp. 1–10. [Online]. Available: https://dl.acm.org/doi/10.1145/3276774.3276794

[102] G. Fierro, A. Saha, T. Shapinsky, M. Steen, and H. Eslinger, "Application-driven creation of building metadata models with semantic sufficiency," in *Proceedings of the 9th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*, vol. 1, no. 1. New York, NY, USA: ACM, nov 2022, pp. 228–237. [Online]. Available: https://dl.acm.org/doi/10.1145/3563357.3564083

[103] Event-Driven Architecture. https://aws.amazon.com/event-driven-architecture/. Accessed: 2023-08-07.

LASITHA CHAMARI is a PhD Candidate at the Eindhoven University of Technology, the Department of the Built Environment. She received her Bachelor of Science in Engineering, specializing in Electical and Information Engineering from University of Ruhuna, Sri Lanka in 2014. She received her Master of Science in Electrical Installations from University of Moratuwa, Sri Lanka in 2020. She joined the Information Systems in the Built Environment research group at Eindhoven University of Technology as a PhD Candidate in August 2021. Her research focuses on developing methods for integration of heterogeneous data for data-driven building applications. Her research interests include IoT, smart buildings, Building Information Modelling, Semantic Web technologies and digital twins.

EKATERINA PETROVA is an Assistant Professor of Artificial Intelligence in Construction at the Department of the Built Environment at Eindhoven University of Technology. She received a PhD in Civil Engineering in 2019 from Aalborg University in Denmark. She was also a visiting researcher at the Department of Architecture and Urban Planning at Ghent University in 2018. She joined the Information Systems in the Built Environment research group at Eindhoven University of Technology in March 2021. Her research focuses on the integration of various symbolic and statistical Artificial Intelligence approaches for decision support in performance-oriented building design and engineering. Her research interests include Artificial Intelligence in the built environment, Ambient Intelligence, cognitive approaches, smart buildings, Building Information Modelling and Management, Semantic Web technologies. She has been working on various topics related to the implementation of symbolic and statistical Artificial Intelligence approaches for decision support in the context of sustainable building design, circular buildings, Digital Twins, data-driven smart buildings, etc.

PIETER PAUWELS works as an Associate Professor at the Eindhoven University of Technology, the Department of the Built Environment. He previously worked at the Dept. of Architecture and Urban Planning at Ghent University (2008-2019). His work and interests are in information system support for the building life-cycle (architectural design, construction, building operation). With a lot of experience and knowledge in computer science and software development, he is involved in a number of industry-oriented research projects on topics affiliated to AI in construction, design thinking, Building Information Modelling (BIM), Linked Building Data (LBD), Linked Data in Architecture and Construction (LDAC), and Semantic Web technologies.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/